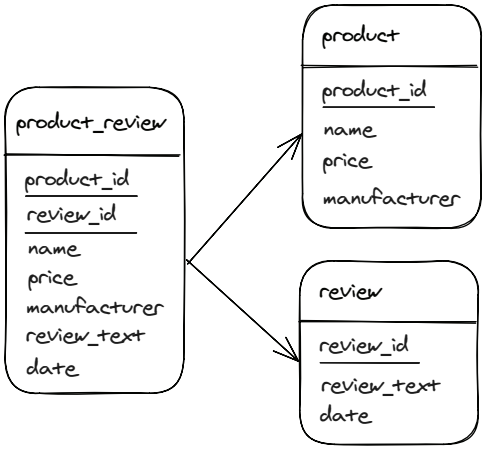


You will learn more about normalization and see how to determine whether a table needs to be normalized in chapter 6.

Implementation/integration and testing

The implementation/integration and testing phase involves building and validating the database based on the blueprint you made in the design and analysis phase. If you use the waterfall approach, the database is implemented all at the same time; if you use the agile approach, the database is implemented part by part and integrated into what has been implemented. Then you test the database to ensure that it functions correctly, performs well, and meets the intended requirements.

During implementation, you create the tables, define the columns and their data types, establish relationships between tables, apply any constraints or rules specified in your design blueprint, and determine which columns to index to optimize query performance. You learned how to use SQL to create a single table or related tables in chapters 1 and 2, and that knowledge can be very useful for this step.

After the database has been implemented, you want to test it before putting it to use. Typically, testing a database needs to validate at least three aspects:

- *Functionality*—You need to check whether the database performs the expected tasks correctly, such as creating, updating, and deleting a data entry.

- *Performance*—You need to check how well the database handles large amounts of data or heavy use.
- *Security*—You need to verify that the database has appropriate security measures in place to protect sensitive data such as passwords and payment methods.

You should identify and fix any bugs discovered during testing, of course. As in the implementation phase, the knowledge of SQL queries you gained from chapters 1 and 2 will be very useful for testing. You will learn more about the details of this phase in chapters 6 and 7.

Key phases of database design

In the preceding section, you got an overview of the process of database design. Starting with this section, you will explore and learn the key phases in database design by designing a database for the online store of The Sci-Fi Collective from scratch. Working on a project from scratch will give you hands-on experience and detailed knowledge of components you would otherwise not pick up.

In this section, you will learn more about the first key phase in database design: requirement gathering. Because requirement gathering is an art rather than a science, following the advice and insights of experts and veterans in this trade can make your life much easier.

The goals of the database

As you take on requirement-gathering tasks, you need to answer a critical question based on all the information to collect: what are the goals of the database?

Every database is created for some specific purpose, whether that purpose is to handle the daily transactions of a business or manage the information of an organization. You need to identify the goal of the database clearly because the database will be used to make important decisions.

Sometimes, it takes longer than expected to come to a full understanding of the goals of the database, and you need to be ready for that situation. To have a good understanding of the goal of the database for The Sci-Fi Collective, you need to interview the owner, managers, staff, and

software developers. You can summarize the goals of this database as follows:

> The database of The Sci-Fi Collective is to maintain information about products, such as their inventory and reviews, and information about users, such as their payment information and purchases, as well as the transaction information linking users and products.

Existing databases and other aspects of information management

Sometimes you can refer to an existing database for your work. If so, you should resist the urge to base your new design on the structure of the existing database. There is a good reason why the organization/business decided to hire you to design a database from scratch instead of modifying the old database. Although the existing database can contain valuable information in terms of what entities and attributes are required to structure some tables, you must be careful about potential design errors in the existing database. Also, you must recognize that it will take the same amount of effort to figure out the demands of the new database and how they differ from those of the existing database.

How do you figure out the current demands? Conduct interviews. How do you figure out the gap between the existing database and current demands? Conduct more interviews with more questions.



With respect to other aspects of information management, you may find that many people discovered ingenious ways to use word processors, spreadsheets, and paper forms to collect and manage data

effectively. If this type of data management needs to be part of the new database you're designing, you may want to do at least two things:

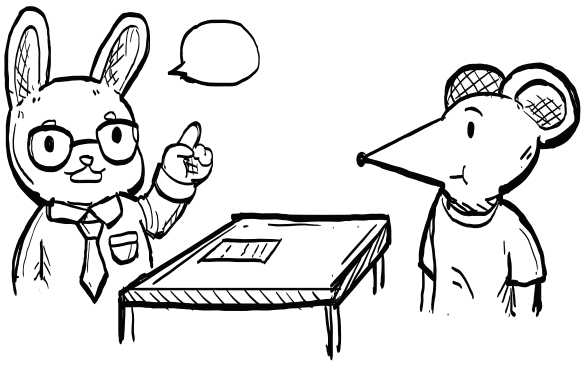
- *Get a subject-matter expert (SME) to walk you through how the data is managed.* This walk-through should involve demonstration and stepwise explanation because it is usually difficult to grasp data management through interviews alone.
- *Ask for data samples whenever possible.* The data samples will play an important role in helping you verify the attributes and data types in the next phase of database design.

We'll use an example to illustrate the preceding two points. The purchasing manager of The Sci-Fi Collective currently manages all the inventory data in a spreadsheet, and the database you design will eventually replace the spreadsheet to manage that data. Instead of talking only to this manager and the purchasing team, you want them to walk you through the whole process of purchasing a batch of products, such as ordering, entering data about a new product, and updating and removing records of products that are already in inventory. The process can be complicated, so getting a demonstration is the best approach whenever possible. More important, you should ask for a copy of the authentic data, which ideally should be anonymized. The data will help clarify many problems that the demonstration can't, such as the exact number of attributes and data types.

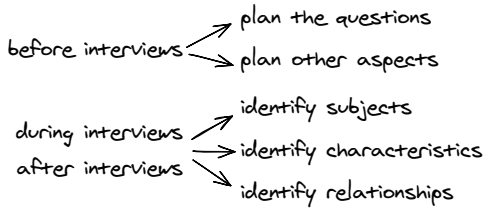
	A	B	C	D	F	G
1	Product name	Manufacturer	Description	Cost	Quantities	MSRP
2	Neuro-Enhancer	CyberDyne Corp	Neural implant ...	$499.99	10	$799.99
3	Quantum Flux...	HyperTech Labs	Revolutionary	$999.99	2	$2,999.99
4	Plasma Blaster	NovaArmory	Portable hand ...	$249.99	15	$599.99
5	Nanotech Gel	BioSolutions Inc	Advanced nano...	$79.99	50	$149.99
...	...	...	...	...	...	...

Interviews

Interviews are the most important tasks in requirement gathering. During and after the interviews, you need to identify three pieces of information: subjects, characteristics, and relationships among subjects. These three pieces of information will be critical to helping you sail through the next phase of database design.



Relationships are self-explanatory, but what are subjects and characteristics? *Subjects* are people, places, things, events, and the like. *Characteristics* are the features of subjects.



Prepare for interviews

Before conducting interviews, carefully plan the questions and other aspects of the interviews. What questions should you prepare? The answer depends on the project as well as whom you are interviewing. To simplify the problem, you may want to group the people you interview. Although the number of groups can vary, a business or organization typically has at least three groups of people you should consider interviewing: stakeholders, SMEs, and IT/technical staff.



The Sci-Fi Collective happens to be a typical business. It has two owners and one manager, two minotaurs who are responsible for tasks such as data entry and customer service, and three elves who work as software developers. Following are some sample interview questions for each group:

- *Stakeholders*

  What is the primary purpose of the database, and what specific goals do you want to achieve with it?

  What key features or functionalities do you expect the database to support?

  Should the database support any specific reporting or analytics needs?

- *SMEs*

  How do you currently manage and organize your data? What challenges or limitations do you face?

  Can you walk me through the typical workflow or process you follow when working with data?

  What specific information or data elements are most critical to your work?

- *IT/technical staff*

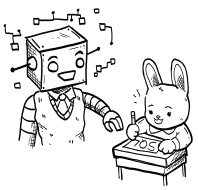
  What are the main tasks or activities you perform that involve data storage or retrieval?

  What reports or outputs do you typically generate from the data? What information do these reports provide?

  Do you perform any specific calculations or computations on the data?

You should prepare more questions for each group yourself. What is the guiding principle for preparing interview questions? A good question should help you gather relevant information about what data the database should store, how the data should be stored, and what constraints should be put in place. If there's no time constraint, however, a bad question is better than no question. If you are new to this process, you can ask for some example questions from ChatGPT. You might ask ChatGPT to provide some sample questions by using a prompt like the following.

> When you need to design a database, you need to conduct interviews with stakeholders. What questions are typically asked during such interviews?



Beyond interview questions, you need to prepare many things beforehand. Here are some key questions that you need to ask yourself during preparation:

- How do you want to record the interview? Will you take notes or record the conversation? If you decide to record, do you need to gain permission from the organization?
- Where do you want to conduct the interviews?
- Do you need to invite more than one group to the same interview? If so, who had better *not* be invited to the same interview? Should there be a limit on the number of interviewees?

Also, ask yourself more questions specific to the project and participants of the interviews. The guiding principle of any preparation is to make the interviews productive and informative.

Identify subjects, characteristics, and relationships

During and after the interviews, you need to identify subjects, characteristics, and relationships among subjects. In case you wonder why, one of the tasks you will take on during the next design phase is mapping the subjects, characteristics, and relationships to entities, attributes, and relationships among entities.

To identify subjects and characteristics during the interview or from the record of the interview, you can look for nouns in the responses to your questions. How do you differentiate subjects from characteristics?

Typically, if you can build a sentence with two nouns in which one *has* or *owns* the other, the one that is possessed is the characteristic, and the other is the subject. You can put *user* and *password* in the following sentence, for example:

> A user has a password.

The password is possessed by the user, so it is a characteristic of a user, whereas the user is a subject.

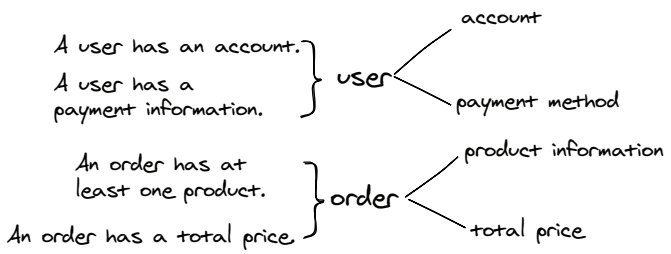
You need to perform similar deductions on interview conversations. You might ask the following question of an IT staff member working for The Sci-Fi Collective:

> Q: What are the main tasks or activities you perform that involve data storage or retrieval?

The participant may give you a response like this (in which all subjects that can be identified are underlined):

> A: As a software developer, I am mainly responsible for building and maintaining the transaction system of the online store. When a user makes an order, the transaction system is supposed to retrieve the user's account and payment method, generate an order with all the information for the ordered products, calculate the total price, bill the payment method, and generate an invoice.

This response is very good. The participant answered your question well, with a lot of useful details. You can track all the nouns and see whether they can be used in sentences with the verb *has* or *owns* to identify the subjects and characteristics:



As you can see, it is easy to deduce at least two subjects—user and order—in this response. A user has two characteristics: account and payment method. An order has two characteristics: product information and total price. Two other subjects—invoice and product—don't have any characteristics.

Typically, you need to ask follow-up questions to get all the characteristics of a given subject. You might have this follow-up conversation with a participant:

> Q: What information does a complete order have?
> A: Well, the same as any other online store: the prices and quantities of all the products bought by a customer. That's it. An order is not complicated.
>
> Q: What about the total price? Is the total price a part of the order?
> A: Sort of. Yes. The total price is calculated based on the unit price and quantities of all the bought products.
>
> Q: What about the date and time when an order was put into the system? Is that a necessary piece of information for an order?
> A: Yes, yes. That's absolutely a necessary piece.

Answers such as "the same as any other online store" and "Sort of" are vague, often requiring you to follow up and ask clarification questions. Luckily, the participant provided enough details after such vague answers. Based on the answers, you can update the characteristics of the order subject as follows:



To wrap up the discussion of subjects and characteristics, you should always ask for sample data if possible. When you have a good understanding of the subjects and characteristics discussed by the participant, you could follow up with a question like this:

> Q: Can you provide some sample data for products, orders, invoices, and users? Anything will help.

After identifying the subjects and characteristics, you will be ready to ask about relationships among subjects. Your focus should be the relationship between every two subjects. You could ask a follow-up question about the relationship between users and orders:

> Q: How are orders related to users? Can a user make multiple orders? Can multiple users contribute to one order?

The participant might give you this response:

> A: A user can of course make as many orders as they like. However, our system doesn't support multiple users contributing to one order.

You could ask a follow-up question about the relationship between products and orders:

> Q: How are orders related to products? Can an order have more than one product? Can a product show up in more than one order?

The participant might give you a response like this:

> A: An order can have more than one product. Vice versa, a product can show up in different orders.

You don't necessarily need to do any analysis of these responses; just record them well.

So far, you have walked through all the necessary steps in an interview. When you finish interviewing all the groups in The Sci-Fi Collective, you will be able to identify a set of subjects and characteristics associated with each subject, as shown in the following figure.

user
* username
* email
* password
* first name
* last name
* phone number
* address

review
* review text
* date and time

product
* code
* name
* description
* manufacturer
* photo
* price
* cost
* inventory quantity

payment method
* name
* credit card number
* expiry date
* billing address

order
* total price
* product price
* product quantity
* date and time

You will also be able to identify the following relationships:

- A user can make multiple orders. An order can be made by only one user.
- A user can review multiple products as long as the user bought those products. A product can be reviewed by multiple users.
- A user can maintain multiple payment methods. A payment method can be associated with only one user.
- An order can have more than one product. A product can show up in multiple orders.

With this information, you are ready to start the next phase in your journey of designing a database for The Sci-Fi Collective.

Recap

- The overall goal of database design is to deliver a well-structured and efficient database.
- Key subgoals of database design include data consistency and integrity, maintainability and ease of use, performance and optimization, data security, and scalability and flexibility.
- All database design approaches have the same key phases, including requirement gathering, analysis and design, and implementation/integration and testing.



- Requirement gathering is the phase in which you gather information about the database in different ways, such as talking to all the people involved in using the database, studying existing databases, and examining other relevant aspects of information management.
- The analysis and design phase focuses on thoroughly understanding all requirements and creating a well-defined plan for the database's structure and functionality.
- The implementation/integration and testing phase is about building and validating the database based on the blueprint you made in the design and analysis phase.
- Conducting interviews is the most important task in requirement gathering. Before interviews, plan the interview questions carefully. During and after the interviews, identify subjects, characteristics, and relationships among subjects.

Part 2
Master database design



Welcome to the heart of database design! This part takes you beyond the basics and into the art and science of creating robust, efficient, and scalable databases. Here, you'll learn how to structure data not just to store it but also to enable powerful, reliable applications.

Chapter 4 begins with entities and attributes—the building blocks of any database. You'll discover how to map real-world objects and concepts to entities and attributes and how to select the right data types.

Chapter 5 focuses on relationships, demonstrating how to create meaningful connections among entities. By the end of this chapter, you'll know how to translate real-world relationships into a coherent, well-structured database.

Chapter 6 introduces normalization and implementation. You'll learn how to refine and strengthen your database designs through normalization and how to translate your designs to SQL, bridging the gap between theory and practice.

Chapter 7 delves into security and optimization. This chapter equips you with strategies to safeguard sensitive data and enhance database performance with techniques such as indexing and denormalization.

By the end of this part, you'll have the expertise to design databases that meet the demands of real-world applications. Whether you're building a small project or tackling a complex enterprise system, this section will prepare you to handle the challenges of modern database design with confidence and precision.

Entities and attributes 4



In this chapter

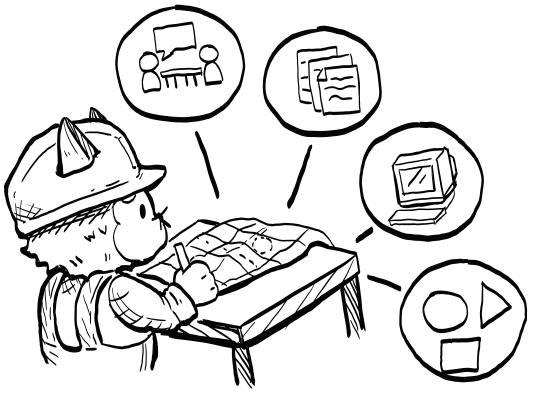
- You start the design and analysis phase of database design.
- You learn about keys and common data types in databases.
- You design your first few entities, identifying their attributes, primary keys, and data types.

What you need to know

You can find the database design covered in this chapter (so far, only entities and attributes) implemented in tools commonly used by practitioners, such as dbdiagram.io and MySQL Workbench, in the GitHub repository (https://github.com/Neo-Hao/grokking-relational-database-design). You can navigate to the `chapter_04` folder and follow the instructions in `README.md` to load the database design into corresponding tools.

The data types covered in this chapter apply to most relational database management systems (RDBMSs), such as MySQL, MariaDB, and PostgreSQL. If you use another RDBMS, such as SQL Server or SQLite, you need to make small modifications to the design by replacing certain data types with equivalents specific to the target RDBMS. You can find such information in `README.md` in the `chapter_04` folder.

Overview



In chapter 3, you walked through the database design process and went over the requirements-gathering phase for the online store of The Sci-Fi Collective.

Starting with this chapter, you will move to the next phase of database design: design and analysis. The first step of design and analysis is data modeling, which aims to generate an entity-relationship (E-R) diagram. In this chapter, you will focus on designing all the required entities for the database of The Sci-Fi Collective. By doing so, you will learn about entities, attributes, keys, and data types.

Entities and attributes

In this section, you will focus on turning the subjects and characteristics you deduced from requirements gathering into entities and attributes. The following figure shows the subjects and characteristics for The Sci-Fi Collective:

user
* username
* email
* password
* first name
* last name
* phone number
* address

review
* review text
* date and time

product
* code
* name
* description
* manufacturer
* photo
* price
* cost
* inventory quantity

payment method
* name
* credit card number
* expiry date
* billing address

order
* total price
* product price
* product quantity
* date and time

Chapter 1 introduced entities and attributes. Now is a good time to refresh your memory of those concepts. An *entity* is a distinct object or concept that can be described by many attributes. A subject and its characteristics may seem ready to be turned into an entity with

attributes with few to no changes. A *subject* is simply an entity, and a *characteristic* is simply an attribute. But you need to put a little bit of thought into naming entities and attributes.

First, you need to choose between singular and plural names for your entities and attributes.

Singular vs. plural: To s or not to s?

Edgar Codd at IBM developed the first RDBMS in the 1970s. In his database, he used singular names for entities (such as `employee`). Other developers followed his lead.

Singular names are best used with primary entities (such as a single `employee` table). Singular approaches have their root in object-oriented programming (OOP), in which a class translates as an entity that contains several objects of the same class.

On the other hand, plural names are more natural as table titles. An `employees` table contains records of employees, for example. Plural table names, however, might cause confusion and errors for someone who isn't sure whether to use plurals in writing queries. Our best recommendation is to aim for consistent use of either convention.

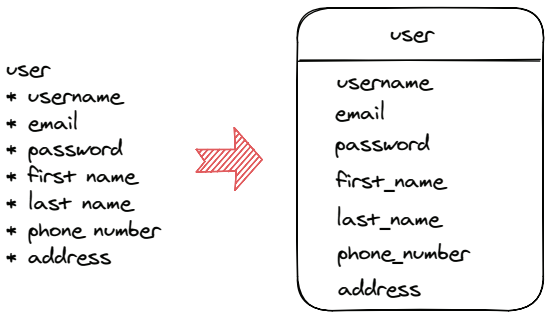
Second, you need to pick a naming convention for the attributes and stick to it. Sticking to a consistent naming convention can prevent typos and other errors in query writing and database maintenance. In this book, we will follow the singular naming convention.

Common naming conventions include

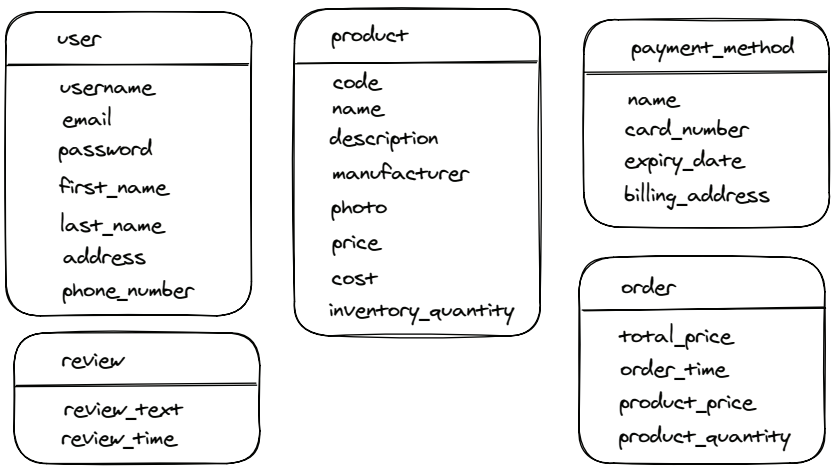
- Snake case (`first_name`)
- Camel case (`firstName`)
- Pascal case (`FirstName`)
- Uppercase (`FIRST_NAME`)
- Hungarian notation, a special case that includes the data type (often abbreviated) as part of the name (`strFirstName`)
- Semantic naming, a special case that includes the purpose of a variable as part of the name (`customerName`)

Among these naming conventions, snake and camel cases are equally popular, followed by Pascal case. Make your choice based on preference and unique software requirements. In this book, we will stick to snake case.

With these two naming decisions made, you can easily map subjects/characteristics to entities/attributes. The `user` subject and its characteristics, for example, can be mapped to the `user` entity and its attributes:



Following the two naming conventions, we will convert all the subjects/characteristics for the online store of The Sci-Fi Collective to the following entities/attributes:



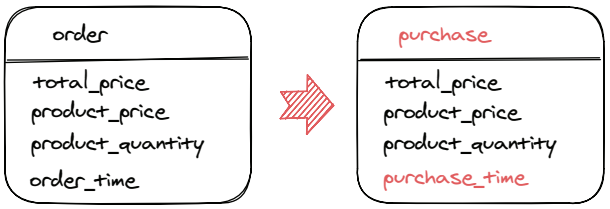
Beyond the naming conventions, you need to check two things about column names:

- *Whether you have names longer than the upper limit*—Many RDBMSs have limits on column-name lengths. MariaDB, for example, limits column names to 64 characters. If you have such a column name, you should shorten it.
- *Whether you used reserved SQL keywords as entity or attribute names*—Using reserved SQL keywords as names can lead to many problems, such as syntax errors in SQL query execution and maintainability problems.

If you use `SELECT` to name an entity, for example, the database system may not understand whether you are referring to the keyword `SELECT` or the entity with that name. Therefore, if you used any reserved keywords as names, you should replace them.

The reserved SQL keywords may vary from one database system to another. You can find a list of keywords in the MySQL documentation (https://mng.bz/zZ6r).

All database systems have a set of common keywords, such as `SELECT`, `ORDER`, `INSERT`, `GROUP`, and `JOIN`. We used the reserved SQL keyword `ORDER` to name one of our entities, so we need to replace it with a different word that has a similar meaning, such as `purchase`:



Reserved keywords in SQL

Reserved keywords in SQL have specific meanings and functions. They define, manipulate, and control the database structure and data. The list of reserved keywords can vary depending on the RDBMS, but the most common keywords are used in all RDBMSs.

If you want to know the common reserved keywords, the fastest approach probably is to ask a generative AI tool such as ChatGPT:

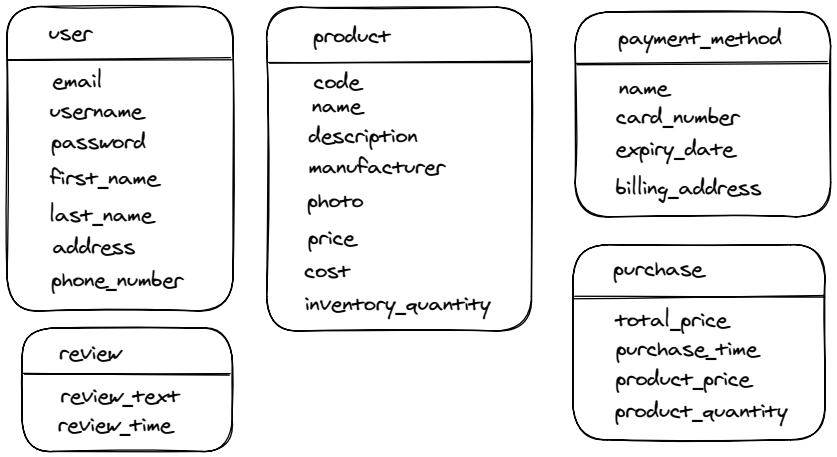
(*continued*)

- What are the common reserved keywords in SQL?

If you need to know the reserved keywords of a specific RDBMS, asking a generative AI tool can still be a good start. To find the reserved keywords in MySQL, you can ask ChatGPT the following questions:

- What are the reserved keywords in MySQL?
- Where are the reserved keywords of MySQL listed in its manual?

When you finish all the preceding steps, the entities of The Sci-Fi Collective's online store will look like the following figure:



Keys

Now that you have converted the subjects and characteristics you deduced in chapter 2 to entities and attributes, you are ready to start identifying primary keys in each entity.

In chapter 1, you learned about primary keys. A *primary key* refers to one or more attributes that can be used to identify an individual data record. The values of primary keys are unique. A table can have only one primary key.

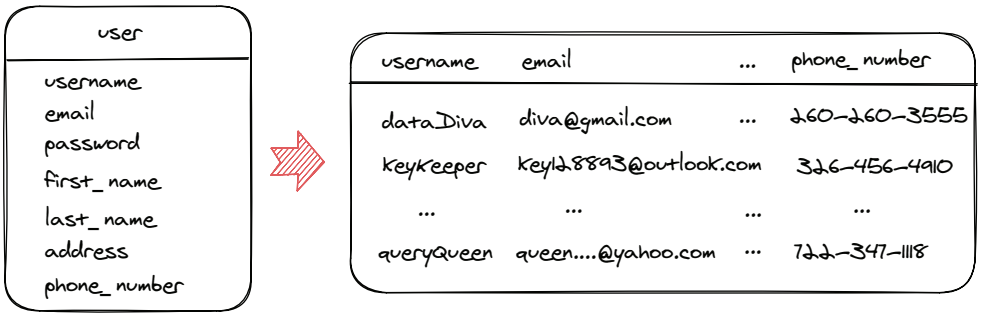
Why does an entity need a primary key? An entity (set) will eventually be converted to a table in a database. The primary key identifies each

row in a table uniquely, enforces data integrity by preventing duplication, and establishes relationships between tables in a relational database. All in all, identifying primary keys is an important step in completing your database design.

The guiding principle in identifying a primary key is simple: pick the best *candidate key* as the primary key. If no good candidate key is available, create a new attribute to serve as the primary key. In this section, we explain what candidate keys are and demonstrate this principle in two examples, starting with the `user` entity.

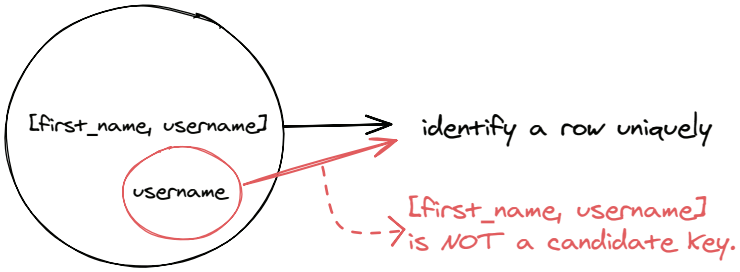
The `user` entity contains seven attributes. When you put the gathered requirements and sample data side by side, you can easily spot the attributes that should have unique values, preventing duplication of user data:

- `username`—Two users can't have the same username. A new user can't register with a username that's already in the database.
- `email`—Emails must be unique for verification and account recovery. An email address can't be used to register two different users.
- `phone_number`—Phone numbers must be unique for verification and account recovery. Different users can't register the same phone number.

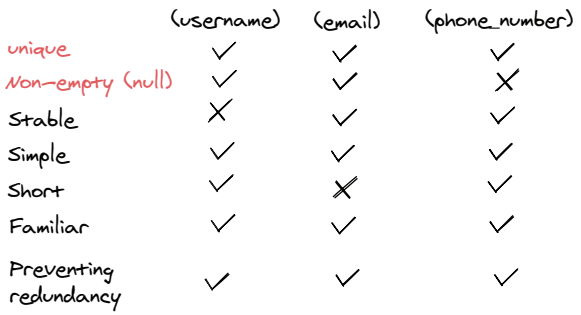


First things first. What is a candidate key? The three attributes are three different candidate keys for the `user` table. A *candidate key* is the smallest possible combination of attributes that can uniquely identify a row in a table. *Smallest* means that no subpart of a candidate key can uniquely identify a row. The combination of `username` and `first_name`, for example, can identify a row uniquely, but `username`, when used as a

subpart of this combination, can also identity a row uniquely. Therefore, the combination of `username` and `first_name` is not a candidate key. On the other hand, `username` is a candidate key.



Can you take a systematic approach to identifying candidate keys? Yes. You start by examining each attribute to see whether it can identify a row uniquely. If you can't find such an attribute, start combining columns to see whether they can identify a row uniquely. In our case, the `user` table contains three candidate keys: `username`, `email`, and `phone_number`. To pick one primary key, you must compare their qualities:



The preceding list is not exhaustive but serves as a general guideline. The meanings of these requirements are clear. *Stable*, for example, means not likely to change, and *simple* means easy to understand and use. The *unique* and *nonempty* (non-nullability) requirements are satisfied almost by default in `username` and `email`. The `phone_number`

values might be `NULL` because even if a customer doesn't have or doesn't want to give us a phone number, we still welcome that customer to register as a user.

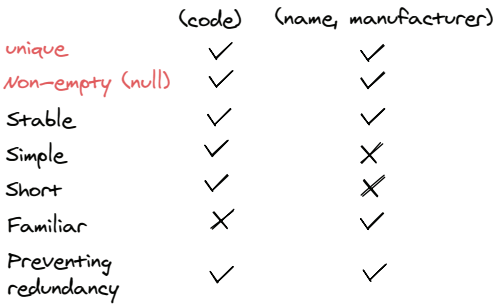
The rest of the metrics can be quite subjective. Usernames, for example, can be easier to change than email addresses; on the other hand, email addresses tend to be longer, which affects access speed. In our case, we will settle on `email` as the primary key for the `user` table because of its stability. The Sci-Fi Collective allows users to update their usernames but not their email addresses, and updates to primary key values are strongly discouraged. You can indicate which attribute is the primary key by underlining it or adding a key symbol to its left:



Consider another example. Among the seven attributes and their possible combinations in the `product` table, you can identify at least two candidate keys:

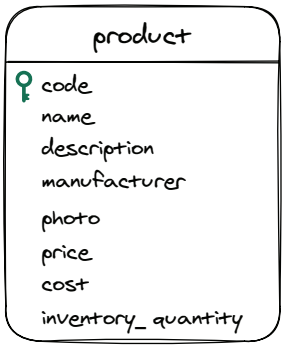
- *Product codes*—In the real world, most products have product codes, which can help identify products uniquely. Because each product has a unique code, the code can also prevent redundancy.
- *Product names and manufacturers*—The combination of product names and manufacturers can also uniquely identify products and prevent redundancy in the `product` table.

The following figure compares these candidate keys:



The product code is a clear winner over the combination of product name and manufacturer. The product code is simpler and shorter.

Although the concept of the product code may be less familiar, it is not difficult to grasp. The product code is based on the Universal Product Code (UPC), unique 12-digit numbers assigned to each product sold in stores and online around the world. So you can settle for `product_code` and use it as the primary key for the `product` table:



In both examples, we picked the primary key from a set of candidate keys. What if no candidate key is available or is a good fit? We will answer this question by picking a primary key for the `review` table, which has two columns: `review_text` and `review_time`. Neither of the two columns can uniquely identify a row in the `review` table. Although the combination of `review_text` and `review_time` can identify rows uniquely in most cases, it is still possible for two reviews with the same text to be recorded at the same time. That said, no candidate key is available for the `review` table:

	(review_txt)	(review_time)	(review_txt, review_time)
unique	✗	✗	✗
non-empty (null)	✓	✓	✓
Stable	✗	✓	✗
Simple	✗	✗	✗
Short	✗	✗	✗
Familiar	✓	✓	✓
Preventing redundancy	✗	✗	✗

When you are in a situation like this one, always think about the alternative: creating a new column and using it as the primary key. You can opt to create a numeric column and use it as the primary key for the `review` table. Numeric primary keys are smaller and can fit completely in computer memory, making them much faster to search. They also scale better and are more standardized than a combination of two attributes. We will create an autoincrementing numeric attribute, `review_id`, and use it as the primary key for the `review` table. This type of key is known as a *surrogate key*.

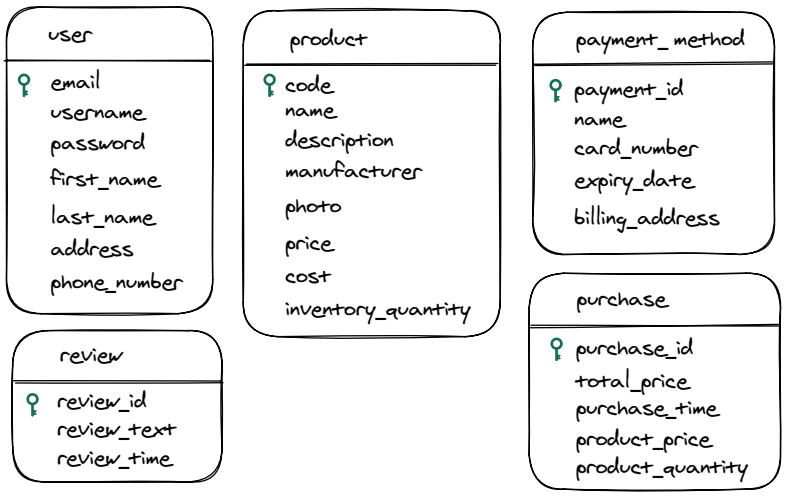
Undoubtedly, a surrogate key can identify rows in a table uniquely. But can such a key prevent redundancy? Well, not by itself. It is still possible for two identical rows of data to be inserted into the `review` table, as shown in the following figure:

review_id	review_text	review_time
...	...	...
7	"Great product"	"9/14/2024, 9:15:15 AM"
8	"Great product"	"9/14/2024, 9:15:15 AM"

Would redundancy be a problem? For tables such as `user` and `product`, it is important to eliminate redundancy. Think about maintaining two rows of data that represent the same user. Which row should be used when the user updates their username? By contrast, reviews are always displayed as lists of texts on the same screen. As long as a row of review data can be identified uniquely, it can be updated or deleted properly. Therefore, redundancy won't be much of a problem for the `review` table.

Following the same logic, you see that the `payment_method` and `purchase` tables are in a similar situation: identifying each row uniquely is more important than eliminating redundancy. For the `purchase` table, redundancy is tolerable. The `purchase` table represents the transaction records of users. A transaction record is immutable. For the `payment_method` table, redundancy is not only tolerable but also acceptable. The Sci-Fi Collective allows two different users to add the same payment method to their individual accounts. (Think about when a couple uses each other's credit cards.)

In such a scenario, autoincrementing surrogate keys are a great choice for the primary keys. We will name the surrogate keys `payment_id` in the `payment_method` table and `purchase_id` in the `purchase` table. The following figure shows the primary keys of all entities:

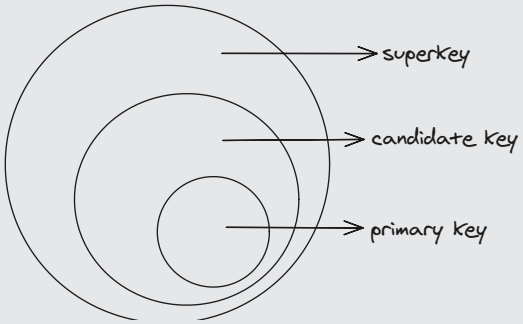


Superkeys, candidate keys, and primary keys

Another type of key that we haven't mentioned yet is the superkey. A *superkey* is a set of one or more columns of a table that can uniquely identify a row in the table. But—shame on the superkey—it may contain columns that aren't required to uniquely identify a row.

Now you know what superkeys are, you can see candidate keys from a different angle: a candidate key is a minimal superkey, which means that it has no unnecessary columns.

Finally, of all the candidate keys in a table, one is chosen as the primary key. The following figure shows the relationships among the three types of keys:



Data types

You successfully identified or generated the primary keys of all entities in the preceding section. In this section, you will work to define data types for all attributes—a necessary step toward completing your design of entities and attributes. To implement the database based on your design, you need the data types of attributes.

Data types are not an entirely new topic. In chapters 1 and 2, you saw how data types are used in SQL queries. In chapter 3, you were encouraged to get sample data that helps you figure out the data types of the characteristics of subjects in the requirements-gathering phase. In this section, you will take advantage of the sample data you accumulated and use it to define data types of all attributes accurately. Meanwhile, you will pick up some new conceptual knowledge of data types.

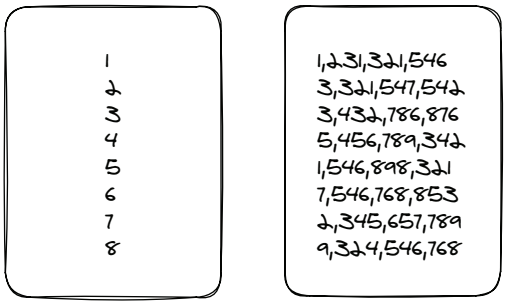
String data types: Power of personality

String data types are among the most-used data types in databases. Before you use string data types, you will learn a little bit about their variations.

CHAR, VARCHAR, and TEXT

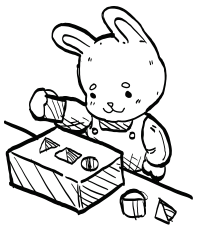
The most common types of string data are `CHAR`, `VARCHAR`, and `TEXT`. The major difference among them lies in storage requirements. (Some require more bytes than others due to how they are physically implemented, for example.)

You may wonder why bytes matter. After all, people have terabytes of available space on their disks. When databases are small, the result is negligible, but as databases grow, so do computational (such as access speed) and memory requirements. Here's an example that shows the effect of data length on search speed:



Please try finding the number 8 in the left figure; then try finding 1,546,898,321 in the right figure. If it took you even a millisecond longer to find the longer number, congratulations: you are no different from a computer. Your choice of data types may cause data to bloat, which in turn slows data lookups (such as `SELECT`) and other operations.

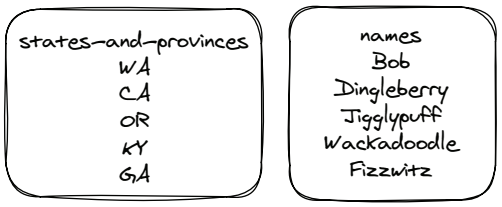
Now that you know that bytes matter, we can present some simple guidelines for choosing a string data type:



- When an attribute has data of uniform length, use `CHAR`.
- When an attribute has data of a relatively short but varied length, use `VARCHAR`. *Relatively short* here also implies that an upper limit can be easily estimated (such as 500 characters).
- When an attribute is expected to have a large amount of text data that regularly exceeds a few thousand characters (1,000 plus), use `TEXT`, especially if the upper limit is hard to estimate.

Some examples can help demonstrate these guidelines. If you need an attribute to store state/province data as part of the address information of your customers in the United States and Canada, you can use

a two-character designation (such as WA for Washington state). CHAR is the perfect data type for this attribute. As another example, if you need an attribute to store users' names, you can expect data length to vary. Also, you can estimate the upper limit of the names; most names don't need to go beyond 30 characters. Therefore, VARCHAR is the perfect data type for this attribute. The following figure summarizes these two examples:



In the preceding figure, you may notice that CHAR and VARCHAR are followed by parentheses with numbers in them. This syntax deserves some explanation. When you use CHAR or VARCHAR to define data types, you need to declare the length. CHAR(2) indicates that each data record of this attribute will occupy exactly 2 bytes. VARCHAR(30) indicates that each data record of its attribute can occupy up to 30 characters. In some RDBMSs, such as MySQL, a record can be rejected or truncated when its character length exceeds the declared length.

As for the data type TEXT, it is typically used to hold a large amount of text data, such as a product description, a product manual, or book text.

Strings and database performance

In some databases, TEXT may not be stored inline along with the rest of the columns; instead, the data may exist in secondary file storage, requiring further lookups and a performance delay. Use this data type sparingly and only when necessary (for description fields, for example).

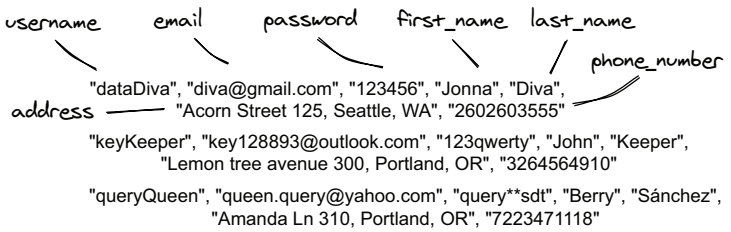
Also, text attributes are indexable through full-text indexes (specialized indexes for larger text that work like a search engine), which are not supported by all database systems. Before version 5.6, for example, MySQL had no support for this feature in its InnoDB engine. In such cases, all searches on text attributes have to be linear and therefore will be slow. If the string data in an attribute is short, and if you expect this attribute to be searched frequently, VARCHAR is the better choice of data type.

Identify string attributes in our database

Now that you have learned the basics of string data types, you are ready to identify the attributes in our database for which string data types are a good fit. To complete this task, you will do the following for each entity:

1. Check the requirements and the sample data that you got from the requirements-gathering phase.
2. Identify the attributes that should be strings.
3. Choose a string data type (`CHAR`, `VARCHAR`, or `TEXT`).

First, you check the requirements and sample data. Our `user` entity, for example, has seven attributes. The following figure shows the sample data for this entity:

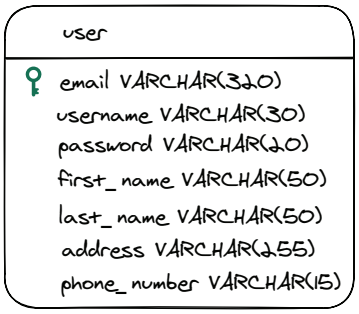


Next, you identify attributes that should be strings. From the sample data, you can tell that all the attributes are strings. In other words, all seven attributes are string data types.

Finally, you choose a string data type for the identified attributes. From the sample data, you can tell that all attributes have variable lengths, and it is not difficult to estimate the maximum length for these variables. Therefore, `VARCHAR` is the most appropriate data type. A phone number, for example, cannot contain more than 15 digits (excluding formatting) no matter where you are, thanks to the international phone numbering plan. Therefore, the data type of `phone_number` would be `VARCHAR(15)`.

You may wonder whether storing the passwords as plain text using `VARCHAR` would introduce a security problem. Of course it will! We will address security-related challenges in chapter 7. For now, `VARCHAR` is good enough.

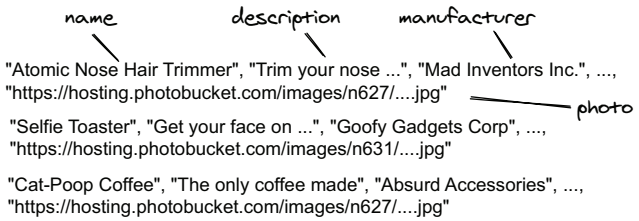
When you identify all the data types of the attribute, you can put the data type to the right of the attribute to complete your design of the `user` entity:



In case you have difficulty estimating a reasonable character limit for a `VARCHAR` attribute, always ask the domain experts or developers follow-up questions. When you don't have access to them, at least consult ChatGPT. A question like the following can help you learn a lot:

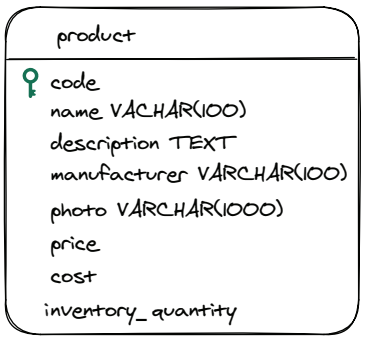
> What's the maximum length of common names?``

Now let's turn our focus to the `product` entity. The following figure shows the sample data for the `product` entity:



From the sample data, you can tell that all four attributes are strings. `VARCHAR` is good enough for the `name` and `manufacturer` attributes. But you can expect product descriptions and photo URLs to be long strings—especially product descriptions. To be safe, you can assign `TEXT` to the description attribute. Considering that the photo URLs

won't be unreasonably long, you can stick to `VARCHAR` but declare a length that can accommodate URLs up to 1,000 characters. The updated `product` entity looks like this:



Lost in translation: Character sets

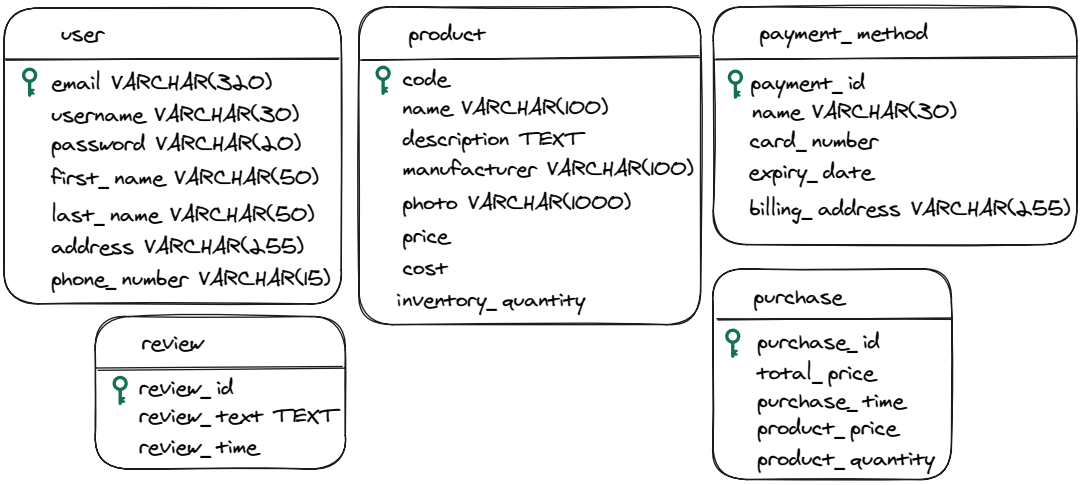
In every database you design and implement, you need to make a big decision about the tables, columns, and data in it. That decision is which character set to use for a database, and it's better to make it early than late.

A *character set* is a set of rules determining how characters are represented and stored in a database. You can think of a character set as being a set of guidelines that a database system follows to handle letters, numbers, symbols, and other characters from various languages and scripts.

Why do you need to decide on a character set for a database? Think about human languages around the world; different languages use different letters and characters. English uses the letters A, B, C, and so on, whereas Chinese uses characters that look very different. Different character sets support different languages. There are character sets for Latin-based languages (such as Latin1), as well as character sets for internationalization and multilingual content (such as UTF-8). When you design a database, it's important to choose a character set that aligns with the languages and symbols you'll be working with to ensure data accuracy and compatibility.

You typically decide on the character set when you create a database is created, which is why it's best to make the decision early. Some RDBMS may not allow you to change the character set, and even if you can, changing character sets can be complex and may result in data loss or corruption.

What about the other entities? Feel free to pause reading and try to identify and define their string attributes. If you are pressed for time, here are the declared string attributes for all entities:



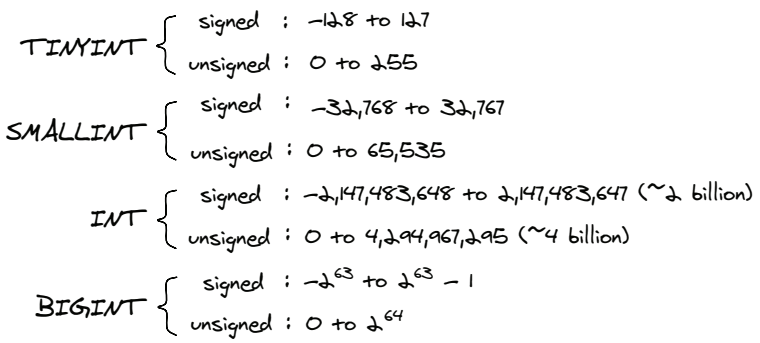
Integers: Number your way to success

Before you can use integer data types fluently, you need to learn a little bit about them.

TINYINT, SMALLINT, MEDIUMINT, INT, and BIGINT

The most common types of integer data are `TINYINT`, `SMALLINT`, `INT`, and `BIGINT`. These data types allocate a fixed number of bytes based on a power of 2 and are 1, 2, 4, and 8 bytes, respectively. Also, these data types have different minimum and maximum values, which vary depending on whether they are set up as signed or unsigned.

In some database systems, an unsigned number doesn't have a + or – sign associated with it. In other words, an unsigned number cannot be negative. By contrast, a signed number can be negative. The ranges of the four integer types, signed and unsigned, are shown in the following figure.



As you can see, `TINYINT` can hold a maximum value of 255 as unsigned but only 127 as signed. `INT` can accommodate a maximum value as 4,294,967,295 as unsigned, which is big. But if you have a numeric attribute with a data value that can be bigger than 4 billion, you need the help of `BIGINT`.

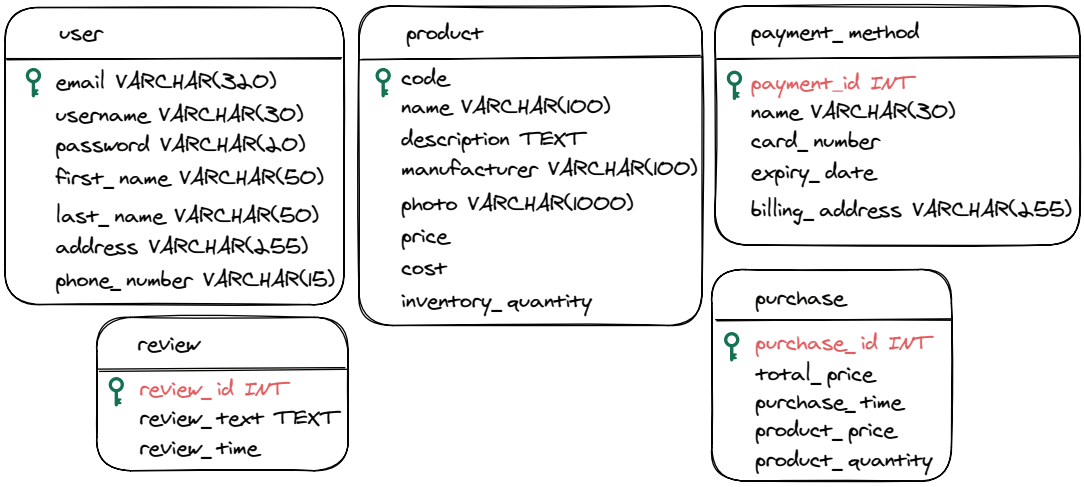
Identify integer attributes in our database

Now that you have learned the basics of integer data types, you are ready to identify the attributes for which integer data types are a good fit in our database.

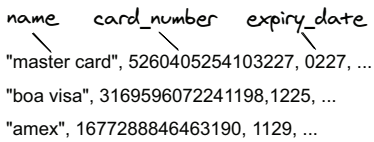
To complete this task, you will follow the same procedure you used to identify string attributes. Start by examining the requirements and sample data that you got from the requirements-gathering phase; then identify the integer attributes and choose an appropriate integer data type for each attribute.

First, focus your attention on the ID attributes. Numeric IDs are stable, unique, simple, and efficient. Following this logic, you can declare the `payment_id` attribute as an integer attribute, for example. You need to decide among the different types of integer data types, of course. It is self-evident that `TINYINT` is too small and thus not a good fit. If you aim to build a super online store that sells products across the galaxy, `SMALLINT` is also too small and not a good fit. `BIGINT` would work well and is big enough. But you can comfortably settle on `INT` because `INT` is big enough for now and yields comparatively better database performance. (The fewer bytes are used, the faster the `SELECT` query will be, for example.) Although whether an integer attribute is signed or unsigned is rarely reflected in an entity-relationship (E-R) diagram, you

still need to make that choice. All primary keys have no reason to have negative values, so they are unsigned.



Most integer attributes in our database are easy to identify, but there are two tricky cases: the `payment_method` entity and the `code` attribute in the `product` entity. Here is the `payment_method` entity in the requirements-gathering phase:

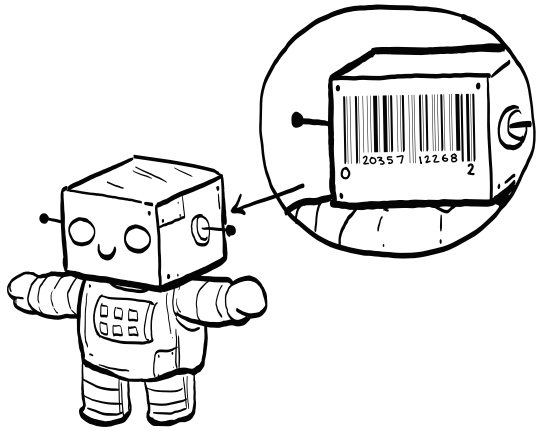


The values of both `card_number` and `expiry_date` seem to be integers in the sample data, but we recommend that you define them as strings instead of integers. This case is one in which you have to sacrifice storage efficiency for clarity, practicality, and correctness. Because no mathematical operations are performed on information such as debit/credit card numbers and expiry dates, you don't need to make the values integers. What's more, making them integers will cause unnecessary problems because a card number or expiry date may have leading zeros. In numerical data types, the leading zeros are eliminated, so such information would not be stored correctly. As a

result, you should define the data type of card numbers as `CHAR(16)` and `expiry_date` as `CHAR(4)`, where 16 is the number of digits: all debit/credit cards have 16 digits, and 4 is the number of digits in every expiry date.

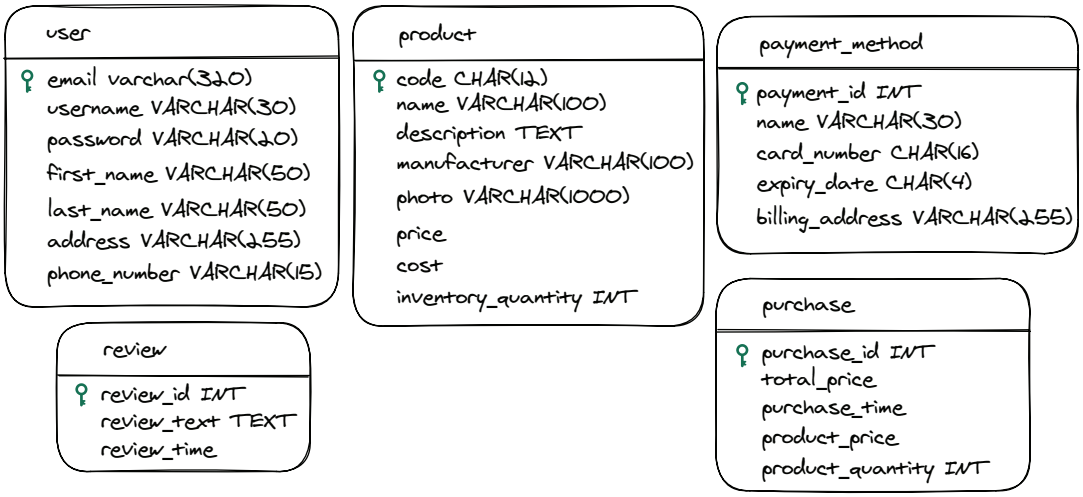
You may wonder whether it's safe to store payment information, such as card numbers, as plain text. Nope. We will revisit the data types of these three attributes in chapter 7, which examines database security. For now, plain-text payment information is good enough when you are still learning about data types.

Your next challenge is the `code` attribute in the `product` entity. The product code is based on the Universal Product Code (UPC), unique 12-digit numbers assigned to each product sold in stores and online.



For the same reasons (leading zeros), it is more appropriate to define the data type of `code` as a string than as an integer. Because all UPC values have the same length, 12 digits, we can define the `code` attribute as `CHAR(12)`.

You can identify other integer attributes and define their data types the same way. You should think about the maximum practical value of giving an integer attribute a specific integer type. This approach will help keep your database small and efficient in the long run. Here's what the remaining integer attributes look like for all entities:



Float like a butterfly, decimal like a data queen

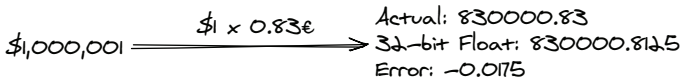
In the preceding section, you may have noticed that we did not assign an integer data type to the `price` attribute of the `product` entity. You may wonder what data types are more appropriate for this attribute and others like it. The answer is decimals. Before you can use decimal data types fluently, you need to learn a little bit about them.

Floating-point vs. fixed-point data types

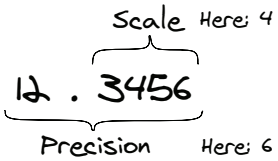
You have two choices for declaring a decimal attribute: *floating-point* types and *fixed-point* types. Common floating-point data types include `FLOAT` and `DOUBLE`. `FLOAT` is typically 32-bit, and `DOUBLE` is typically 64-bit. In other words, `DOUBLE` can accommodate bigger numbers. `FLOAT` and `DOUBLE` are used widely when accuracy is not strictly required, as in sensor reading, statistical analysis, and physical simulation. When accuracy is required to the extent that you should care about every decimal value, you should avoid the `FLOAT` and `DOUBLE` data types due to the need for finite precision and the potential for rounding errors.

Suppose that you have a database that keeps track of money exchanges through different currencies for its users. (Handling other people's money is fun.)

If you want to convert 1,000,001 USD to EUR with an exchange rate of 0.83, rounding errors in a 32-bit floating-point operation may result in 830000.8125. This result is about 1.75 euro cents less than it should be. This difference may not seem like much, but users of such a system will lose money for no reason, especially frequent users and those who transfer huge amounts of money.



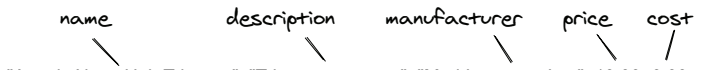
Now that you have a rough idea of what we mean by *accuracy*, let's define the word formally before moving further. *Accuracy* means specific requirements for precision and scale. *Precision* refers to the total number of significant digits, and *scale* refers to the number of digits stored to the right of the decimal point. Consider the number 12.3456: its precision is 6, and its scale is 4.



When accuracy is required, you should think about fixed-point data types such as `DECIMAL` and `NUMERIC`. There are no real differences between `DECIMAL` and `NUMERIC` in many RDBMSs, such as MySQL and PostgreSQL. In such cases, you can use either data type. If you are not sure whether the RDBMS you use falls into this category, refer to its documentation or check with ChatGPT.

Identify decimal attributes in our database

Now that you know the basics of decimal data types, you are ready to identify the decimal attributes in our database. To complete this task, you will follow the same procedure that you used to identify string and integer attributes. To identify the decimal attributes in the `product` table, for example, take a look at the sample `product` data:



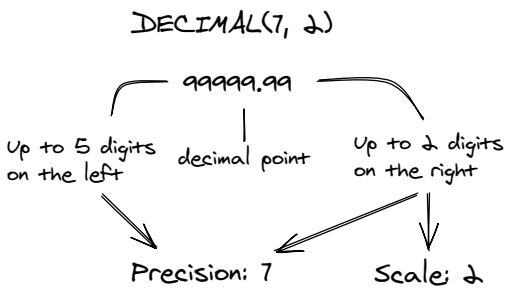

```
"Atomic Nose Hair Trimmer", "Trim your nose ...", "Mad Inventors Inc.", 19.99, 9.99, ...

"Selfie Toaster", "Get your face on ...", "Goofy Gadgets Corp", 24.99, 15.02, ...

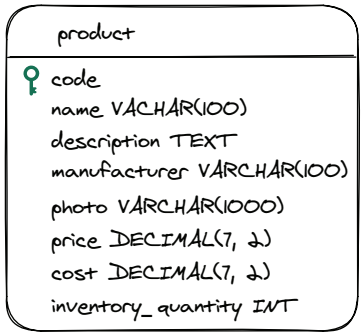
"Cat-Poop Coffee", "The only coffee made", "Absurd Accessories", 29.99, 21.85, ...
```

Do you see the values for the `price` and `cost` attributes? They represent money. In other words, they require accuracy, so you should

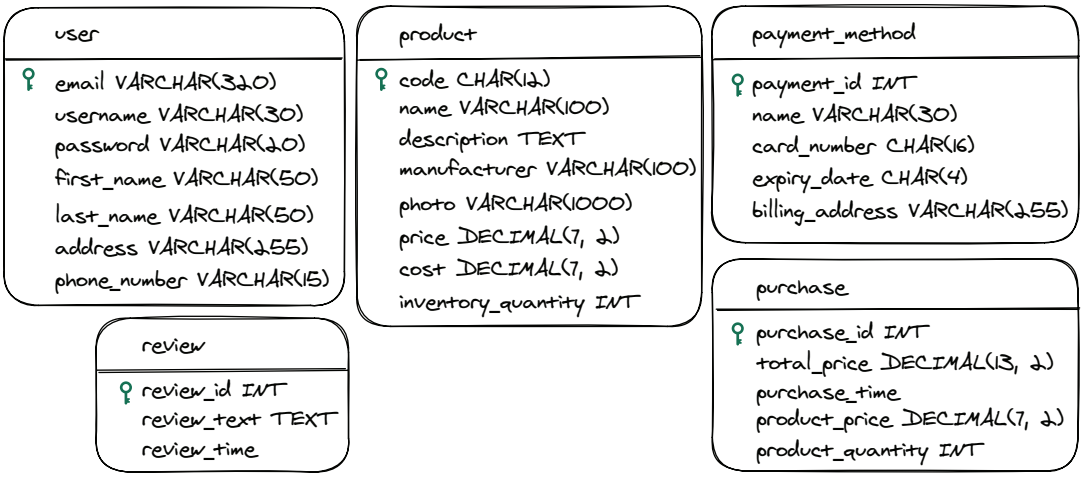
think about using `DECIMAL`. For money, it makes sense for the scale to be 2 because that's the smallest unit of money (cents). In larger monetary operations, a higher scale may be necessary to accommodate for roundoff operations. All our products sell for less than $1,000, so precision can comfortably stay as 7:



Therefore, you define the data type of both the `price` and `cost` attributes as `DECIMAL(7, 2)`, where 7 specifies the precision and 2 specifies the scale. You can update your design of the `product` entity accordingly:



You can apply the same steps to identify decimal attributes in other tables—essentially any other attributes that are money-related, such as the `product_price` and the `total_price` attributes in `purchase`. If you worry that a single transaction may lead to a large bill, you can increase precision for the `total_price` attribute, such as `DECIMAL(13, 2)`. The following figure shows the updated design of all the entities.



Temporal data types: Time flies when you're querying

In the preceding section, you may have noticed that we did not assign any data types to the `review_time` attribute of the `review` entity. You may wonder what data types are appropriate for this attribute and others like it. The answer is temporal data types, which belong to the last group of data types you learned about in this chapter. Before you can use temporal data types fluently, you need to learn a little bit about them.

DATE, TIME, DATETIME, and TIMESTAMP

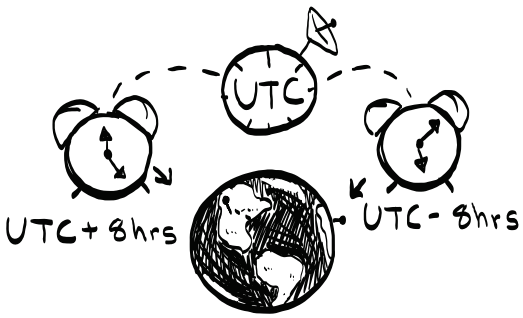
Most database systems support some temporal data types, such as `DATE`, `TIME`, `DATETIM` , and `TIMESTAMP`. Although implementations of such data types vary across RDBMSs, here are some key points that are consistent across most systems and can help you determine which one to use:

- Use `DATE` when a date needs to be stored without any time information, such as birthday data. The date `'1980-05-15'` represents May 15, 1980, for example.
- Use `TIME` when only time needs to be stored. If you want to track the time at which events occur during the day, you can use the `TIME` data type. The time `'14:30:00'` represents 2:30 p.m., for example.

- Use `DATETIME` when both date and time information need to be stored to represent historical time, future events, or other instances in which the time zone is an inherent part of the time and should not be adjusted.
- Use `TIMESTAMP` to record the exact moment of an event that needs to be consistent across time zones, such as log timestamps, record creation, and modification times. Typically, the RDBMS handles the conversion of time zones in `TIMESTAMP` data automatically.

If you are designing a database for an application whose users are strictly local, such as an ordering system for a restaurant, you don't need to worry about time-zone problems. But if you are building an application that might be used around the globe, you may wonder whether the time-zone data will be stored as part of the temporal data attributes. Regardless of the specifics of the RDBMS, you should consider storing date and time values in *Coordinated Universal Time* (UTC) because it ensures consistency and prevents problems with Daylight Saving Time changes, different time zones, and traveling users.

UTC is like a big clock that everyone agrees to follow. It's the same time for everyone, no matter where they are. If you live in Seattle, and your friends in Beijing want to let you know their local time without explicitly giving you the exact time, they can say "Our local time is UTC plus 8 hours." You can not only calculate their local time but also compare their time with yours because you know how your local time compares with UTC. That said, any web or mobile applications can calculate local time given a UTC time.

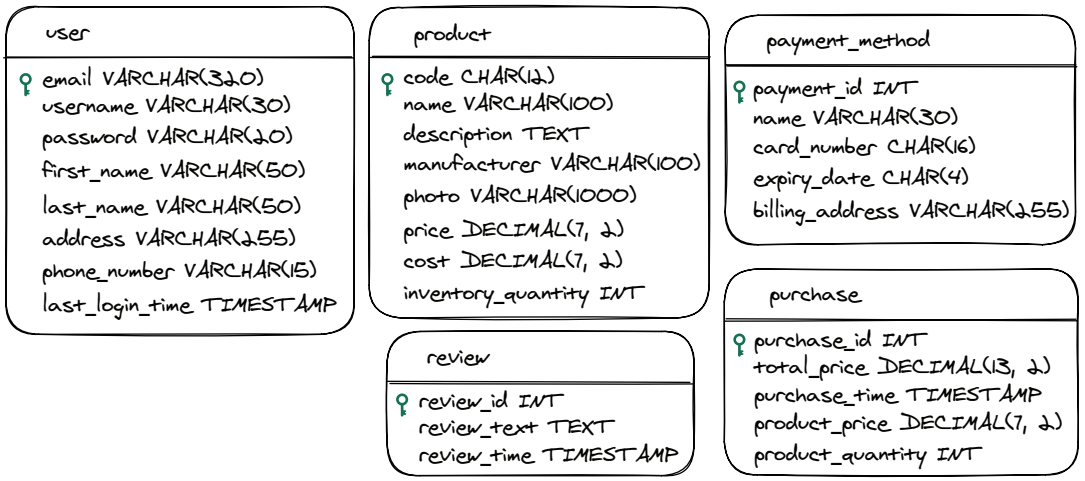


Identify attributes representing date and time in our database

Now you are ready to identify the attributes that represent date and time in our database. To complete this task, follow the same steps you used to declare data types in earlier sections. You start by referring to sample data you collected in the requirements-gathering phase. In this case, however, you may find the task trivial because the attributes related to date and time have self-revealing names (that is, *time* is part of the name).

Two attributes are related to date and time: the `review_time` attribute in `review` and the `purchase_time` attribute in `purchase`. During the requirements-gathering phase, you learned that both attributes require date and time information because they represent exact moments of some events that need to be consistent across time zones. Thus, you can declare the data type of both attributes as `TIMESTAMP`.

Also, you learned that you need to track the last time a user accessed the online store by adding another `TIMESTAMP` attribute to the `user` entity and naming it `last_login_time`. With all these updates, here is another version of the updated entities:



So far, you have successfully identified the data types of all the attributes. Take another look at your masterpiece. You did a great job!

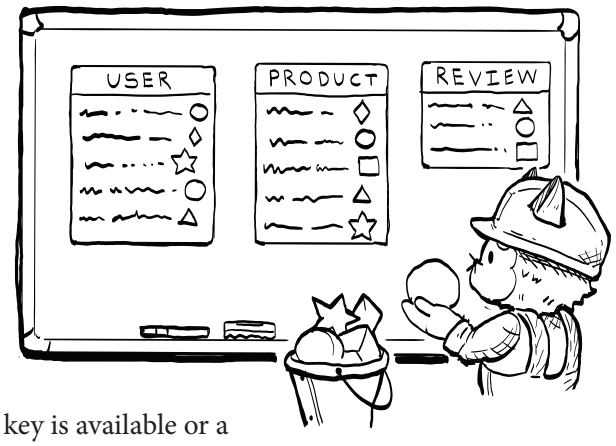
Inconsistencies of data types across RDBMSs

Like it or not, there are inconsistencies of data types across RDBMSs. The data types that you have learned are common across RDBMSs, but you need to learn about exceptions and particularities in individual systems. The entities and attributes you developed in this chapter are perfect for MySQL, MariaDB, and PostgreSQL, but you will need to make small or big adjustments for other RDBMSs for the following reasons:

- In SQLite, it is common to use `TEXT` in place of `TIMESTAMP`.
- Oracle uses `NUMBER` in place of `INT` and `DECIMAL` and `VARCHAR2` in place of `TEXT`.
- SQL Server uses `NVARCHAR(MAX)` in place of `TEXT` and `DATETIME2` or `DATETIMEOFFSET` in place of `TIMESTAMP`.

You can find such information in the GitHub repository that accompanies this book.

Recap



- Designing an entity requires identifying all its attributes, the primary key, and the data types of all attributes.
- A primary key should be able to identify each row in a table uniquely. Depending on your case, you may need a primary key to prevent redundancies. If no candidate key is available or a good fit, you can always create a numeric attribute and use it as the primary key.
- Common data types in databases include string, integer, decimal, and temporal. To decide which data type to use, think about what job a data type is good for as well as the demands of your particular case.

- Given an attribute, to identify its data type, you need to examine the sample data and the information collected during the requirements-gathering phase, identify the proper data-type group (such as string), and choose the best fit within that group (such as `TEXT`) based on the maximum allowed values or spaces derived from your requirements analysis.
- Using proper data types ensures that a database stores data efficiently and meets the demands of data querying.

Relationships 5



In this chapter

- You establish relationships between entities.
- You identify the cardinality of each relationship.
- You decide whether to represent some entities as weak entities.

What you need to know

You can find the database design covered in this chapter implemented in tools commonly used by practitioners, such as dbdiagram.io and MySQL Workbench, in the GitHub repository (https://github.com/Neo-Hao/grokking-relational-database-design). You can navigate to the `chapter_05` folder and follow the instructions in `README.md` to load the database design into corresponding tools.

The data types that show up in this chapter apply to most relational database management systems (RDBMSs), such as MySQL, MariaDB, and PostgreSQL. If you use another RDBMS, such as SQL Server or SQLite, you may need to make small modifications to the design by replacing certain data types with equivalents specific to the target RDBMS. You can find such information in `README.md` in the `chapter_05` folder.

Overview

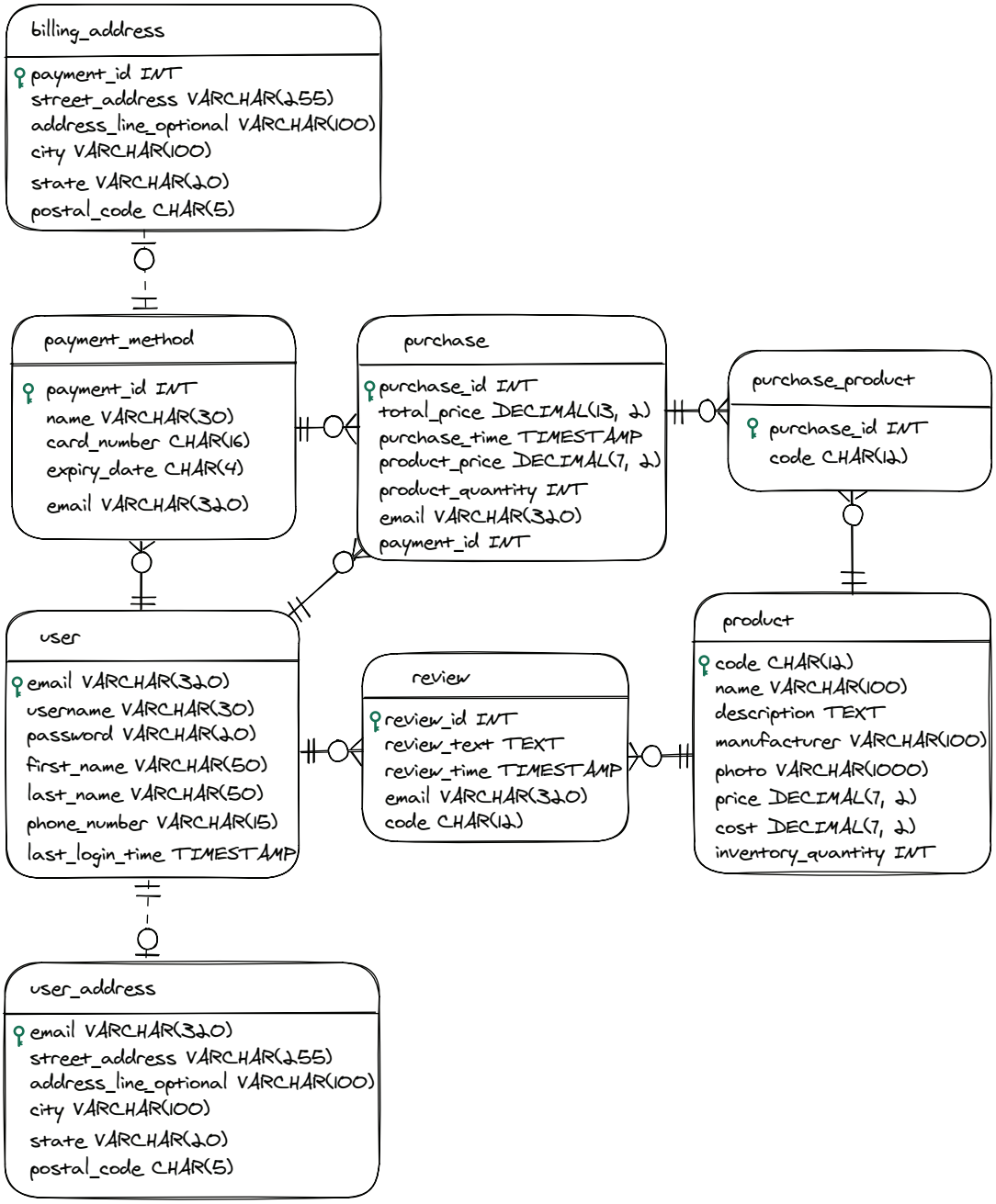
In this chapter, you will develop your entity-relationship (E-R) diagram for The Sci-Fi Collective's online store by establishing relationships among entities you identified in chapter 4. By doing so, you will learn important database design concepts, such as cardinality and dependency.



Entity-relationship diagrams

E-R diagrams are graphical representations of entities and their relationships in a database. E-R diagrams are not only visual tools but also documents that describe database structures and rules. Over the years, these diagrams have become a universal design language among database designers.

An E-R diagram is typically composed of boxes representing the entities and lines representing the relationships among the entities. The diagram depicts the data structure (also known as the *data schema*) but not the data. In an E-R diagram, a table with millions of records is still represented as a single entity. The E-R diagram that you will develop by the end of this chapter looks like this:

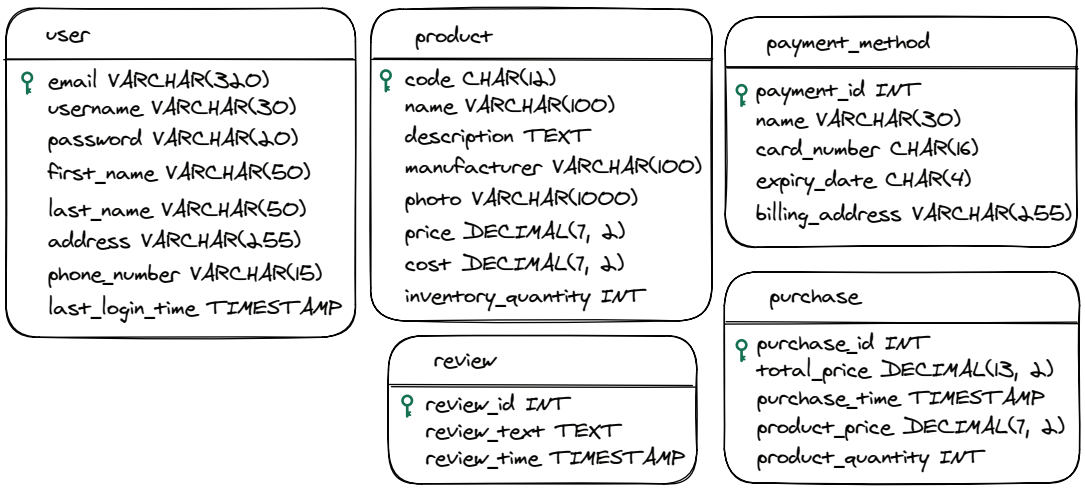

billing_address
payment_id INT
street_address VARCHAR(255)
address_line_optional VARCHAR(100)
city VARCHAR(100)
state VARCHAR(20)
postal_code CHAR(5)
payment_method
payment_id INT
name VARCHAR(30)
card_number CHAR(16)
expiry_date CHAR(4)
email VARCHAR(320)
purchase
purchase_id INT
total_price DECIMAL(13, 2)
purchase_time TIMESTAMP
product_price DECIMAL(7, 2)
product_quantity INT
email VARCHAR(320)
payment_id INT
purchase_product
purchase_id INT
code CHAR(12)
user
email VARCHAR(320)
username VARCHAR(30)
password VARCHAR(20)
first_name VARCHAR(50)
last_name VARCHAR(50)
phone_number VARCHAR(15)
last_login_time TIMESTAMP
review
review_id INT
review_text TEXT
review_time TIMESTAMP
email VARCHAR(320)
code CHAR(12)
product
code CHAR(12)
name VARCHAR(100)
description TEXT
manufacturer VARCHAR(100)
photo VARCHAR(1000)
price DECIMAL(7, 2)
cost DECIMAL(7, 2)
inventory_quantity INT
user_address
email VARCHAR(320)
street_address VARCHAR(255)
address_line_optional VARCHAR(100)
city VARCHAR(100)
state VARCHAR(20)
postal_code CHAR(5)

This diagram uses Information Engineering notation, commonly known as Crow's Foot notation; we will briefly cover other notation types later in the chapter. If you are designing a database for a sophisticated application, you may need multiple E-R diagrams to describe its structure fully. For The Sci-Fi Collective's online store, one E-R diagram is good enough.

Now that we have introduced E-R diagrams, let's start developing the diagram for The Sci-Fi Collective. We are not starting from scratch because we developed a set of entities in chapter 4, but we haven't represented the relationships of those entities yet. In the next few sections, you will learn more about relationships between entities and take on the task of establishing relationships among entities for The Sci-Fi Collective.

Connect related entities

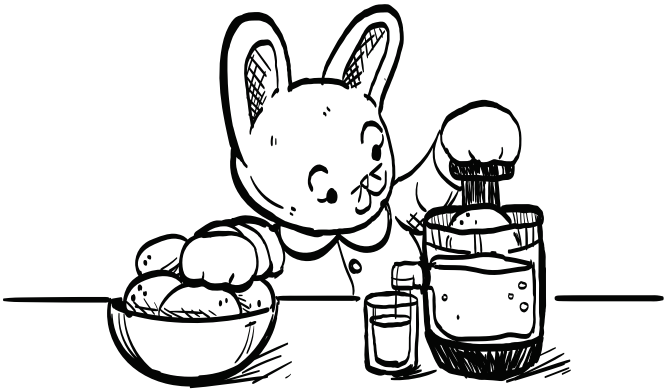
Now is the perfect moment to reflect on and revisit all that you have accomplished so far. In chapter 4, you established all the entities as follows:



In chapter 3, you went over the requirements-gathering phase and collected useful information that pertains to the relationships among the preceding entities. You copied this information:

- A *user* can *make* multiple *purchases*; an order can be made by only one user.
- A *user* can *review* multiple *products* as long as the user bought those products; a product can be reviewed by multiple users.
- A *user* can *maintain* multiple *payment methods*; a payment method can be associated with only one user.
- A *purchase* can *have* more than one *product*; a product can show up in multiple orders.

In an E-R diagram, you use a line to connect every pair of two entities that are related. To establish relationships between two entities, you will identify every relationship and connect every pair of related entities using lines.

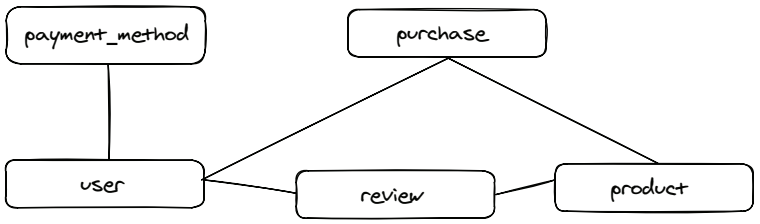


Your first step in connecting entities is synthesizing the information you gathered and the design of entities. Your goal is to generate a list of simple sentences composed only of subjects, verbs, and objects. The sentences will help you understand the relationships between every pair of entities. As you designed entities, you may have introduced changes that conflicted with the gathered information. You designed a `review`

entity, for example, but it didn't appear as a noun in the information you gathered. To reconcile such conflicts, think about whether it makes sense to keep the `review` entity. If so, adapt the synthesized information accordingly:

- A user makes purchases.
- A user writes reviews.
- A product has reviews.
- A user maintains payment methods.
- A purchase contains products.

Next, map this summary to a diagram. The nouns in every sentence represent entities. If two nouns connected by some verbs show up in one sentence, the two entities are+ likely to be related. You may go through a few iterations of trial and error when mapping the summary to a diagram because of possible inaccuracy and misinterpretation. When you draw an entity in your draft E-R diagram, you can skip the attributes for now because they don't matter yet and listing all of them is tedious. Based on the preceding summary, you will develop the following draft diagram:

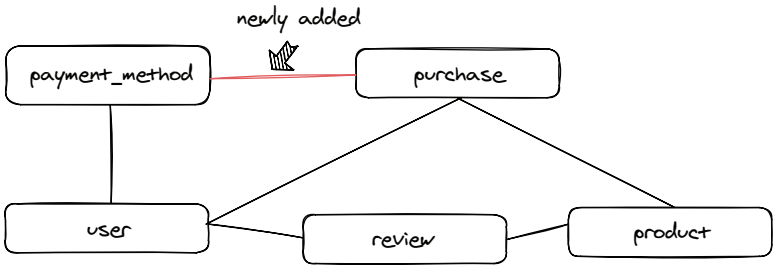


When you generate a draft diagram, you should test every relationship against the information you gathered and the sample data you collected. Also, take the draft diagram to the stakeholders and explain your design rationale to them because it is likely that you made some mistakes or neglected something critical in your first few iterations. The software developers of The Sci-Fi Collective, for example, will point out that an online purchase can't be performed without a payment method. Based

on the new information, you need to answer the following question before revising the draft diagram:

Should `payment_method` be related to `purchase`?

Without the payment method information, an online order can't be finished, and the online store can't bill its users. In other words, each purchase record needs to be mapped to a corresponding payment method. Therefore, a relationship between `payment_method` and `purchase` makes sense. With this question answered, add one more relationship:



In the next two sections, you will learn more about the characteristics of a relationship between two entities, which will empower you to develop the draft diagram further.

Cardinality

Cardinality is an important characteristic of a relationship between two entities, describing the number of instances of one entity that can be associated with a single instance of another entity via the relationship. Based on that definition, cardinality is classified into several types, each of which is represented differently in an E-R diagram.

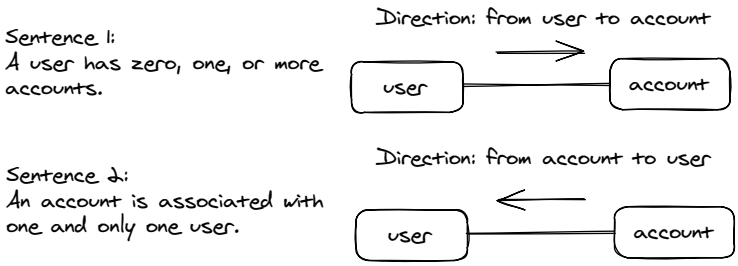
To complete your E-R diagram, you need to analyze the information you collected from the requirements-gathering phase, identify the cardinality of each relationship, and update the draft diagram accordingly.

Direction and representation

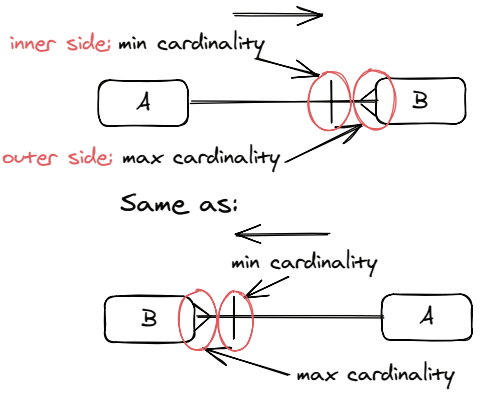
If you consider directions, a relationship between two entities can be broken into two directional relationships. In a banking system, for example, `user` and `account` are two entities, and their relationship(s) can be summarized using two sentences:

- *Sentence 1*—A user has zero, one, or more accounts.
- *Sentence 2*—An account is associated with one and only one user.

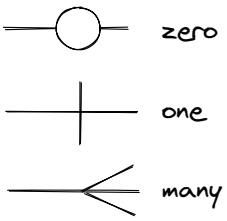
Sentences 1 and 2 represent two different directional relationships between `user` and `account`. In both sentences, the direction flows from the subject to the object:



Given a directional relationship from A to B, cardinality describes the number of instances of B with which a single instance of A can be associated. Cardinality is represented by two graphic symbols on the relationship line between A and B. The symbols are used in pairs to represent the cardinalities. The symbol on the inner side represents the minimum instance number of B that a single instance of A needs to be associated with—the *min cardinality*. The symbol on the outer side represents the maximum instance number of B that a single instance of A needs to be associated with—the *max cardinality.*

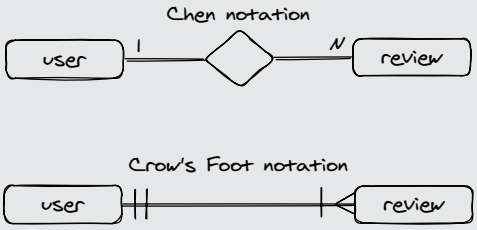


As you see in the preceding figure, to represent the cardinality of the relationship from A to B, you place the two symbols closer to B on the relationship line. In this example, the individual symbols represent quantities. You need to know the symbols for zero, one, and many, shown in the figure to the right.



Cardinality notation: Complicated relationships with math symbols

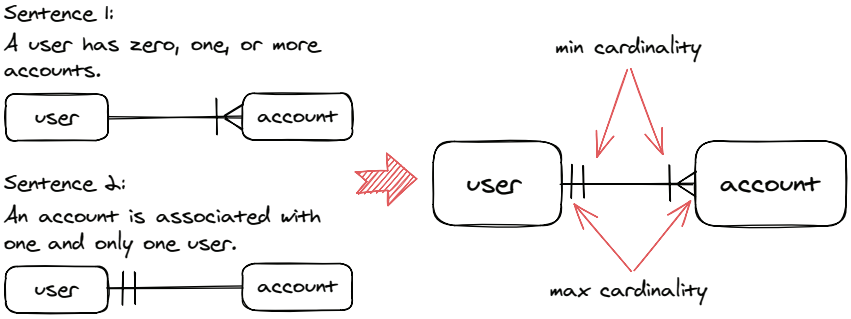
There are several ways to notate the cardinality of a relationship. The two most popular are Chen notation and Crow's Foot notation. Chen notation has historic significance. Crow's Foot notation is simpler and more popular among professionals. In this book, we will stick to Crow's Foot notation.



How do you represent the relationship cardinalities of the example you saw at the beginning of this section—the relationship between the `user` and `account` entities in a banking system?

- *Sentence 1*—A user has zero, one, or more accounts.
- *Sentence 2*—An account is associated with one and only one user.

The two sentences represent two directional relationships; they also contain the information you need to establish their cardinalities. Because a user is associated with zero, one, or more accounts, the min cardinality is zero, and the max cardinality is many for the relationship from `user` to `account`. Similarly, an account is associated with one and only one user, which means that both the max and min cardinalities for the relationship from `account` to `user` are one.

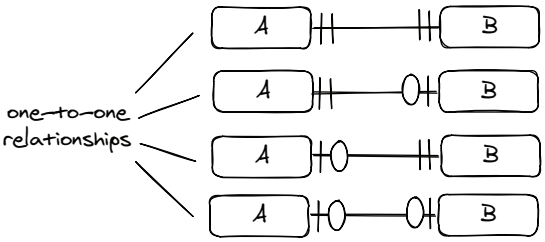


As you see in the preceding figure, you can merge the two directional relationships and use a single line to represent both. The cardinality symbols closer to `account` represent the cardinality of the relationship from `user` to `account`, whereas the symbols closer to `user` represent the cardinality of the relationship from `account` to `user`.

Now you know what cardinality is and how to represent it in E-R diagrams, you will learn about three common cardinality types and apply what you learn to develop the draft E-R diagram further.

One-to-one: A perfect match

In a one-to-one relationship, each record in one entity is related to up to one record in the other entity. *One-to-one* refers primarily to the max cardinality of both directional relationships. The min cardinalities could be either zero or one.

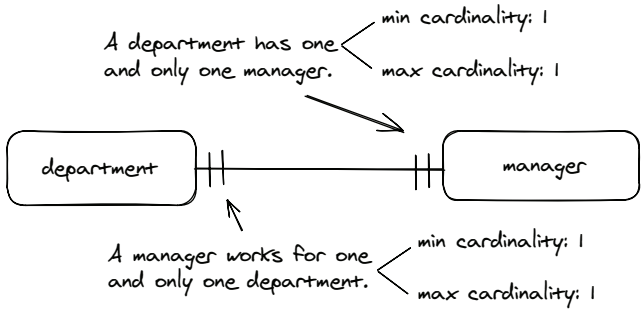


Given a one-to-one relationship, if both of the two min cardinalities are ones, one of the min cardinalities is typically converted to zero for easy implementation.

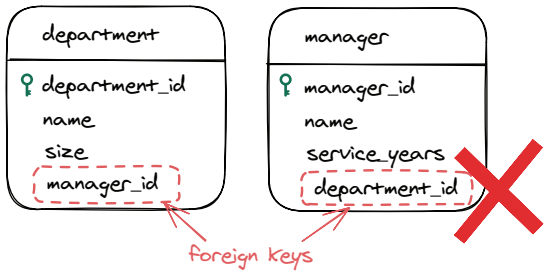
Consider an example. In the database of a large corporation, both departments and managers are represented, and their relationship is as follows:

- A department has one and only one manager.
- A manager works for one and only one department.

You can represent such a relationship in an E-R diagram as follows:



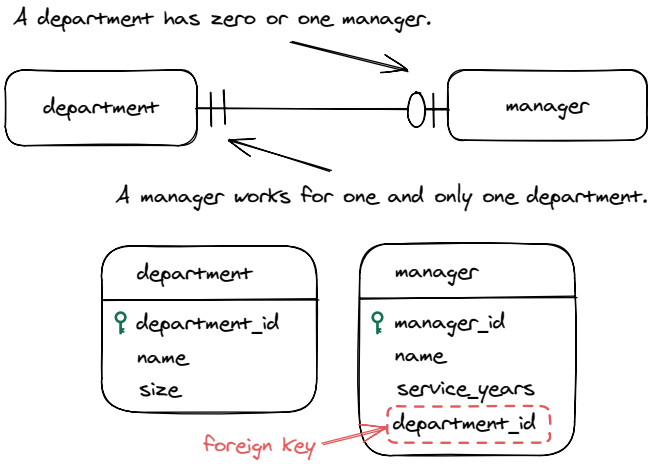
This representation is theoretically solid but impossible to implement. To link the two entities, you need help from foreign keys. If the two entities in the preceding figure have attributes, you need to place foreign keys in both tables because each department is associated with a manager and each manager is associated with a department:



Such an implementation is problematic for data entry or insertion. When the two tables are created, they don't contain data, so you need to start populating the tables by entering `department` or `manager` information. If you enter a `department` record first, its `manager` data won't be available yet. The foreign key constraints you put in place cause SQL to complain and reject the data entry. If you switch the order and enter `manager` information first, you face the same problem. In such a situation, you can relax one of the two min cardinalities from one to zero to make implementation possible. You can modify the relationship between `department` and `manager` as follows:

- A department *has zero or one* manager.
- A manager works for one and only one department.

The representation and implementation are updated accordingly:



As you see in the preceding figure, the min cardinality can indicate where to place the foreign key in a one-to-one relationship. Given a directional relationship from table A to table B, if the minimum cardinality is zero, not every instance of table A must have a corresponding record in table B. In this case, you typically place the foreign key in table B. This change enables data entry. In this case, table A is `department`, and table B is `manager`.

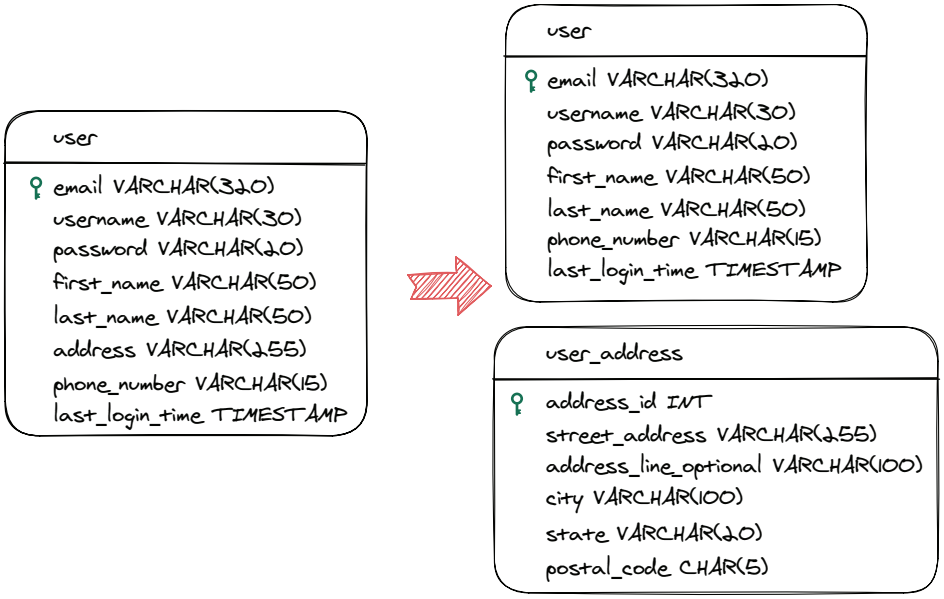
Now you know what one-to-one relationships are, it's time to work on the E-R diagram of The Sci-Fi Collective.

Identify one-to-one relationships in your database

Based on the information you gathered in the requirements-gathering phase, you don't have a one-to-one relationship between the entities that you've identified. But some new information about users' addresses that you learned in the iterative process propels you to redesign your `user` entity. What is the problem? A user's address might look like this:

20 Baldwin Rd, Shelter Island, New York, 11964

The data analysts of The Sci-Fi Collective need to filter users' data based on city, state/province, or zip code to conduct targeted marketing analysis from time to time. Storing users' address information in a `VARCHAR` attribute will make such a task difficult. To address this problem, factor the address information into a different entity with multiple attributes:

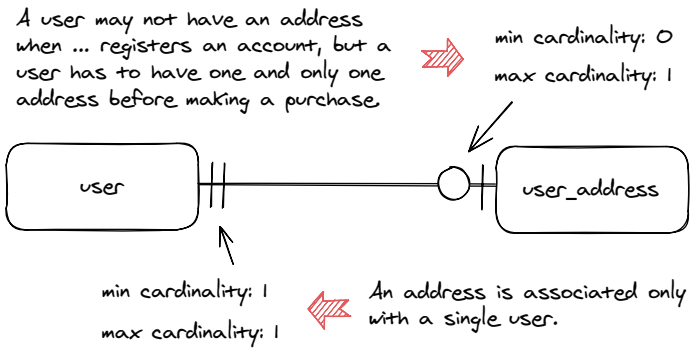


As you can see, most of the attributes of this new entity (`user_address`) are of a string data type. You need to decide whether to assign `CHAR` or `VARCHAR` and their possible max lengths. To simplify the solution, assume that The Sci-Fi Collective operates only within the United States. This assumption will make it easier to determine the maximum lengths of all the string attributes. The `state` attribute, for example, can be assigned `VARCHAR(20)`. North Carolina and South Carolina are among the longest state names (14 characters each). To be safe and accommodate any possible future changes, you can set the `VARCHAR` length 6 characters longer than 14. Or you can assign `CHAR(5)` to the `postal_code` attribute if you opt for the five-digit format.

Now that you have successfully converted the `address` attribute to a new entity, `user_address`, let's focus on the relationship between `user` and `user_address`. Here is the new information you gathered from the developer of The Sci-Fi Collective's online store:

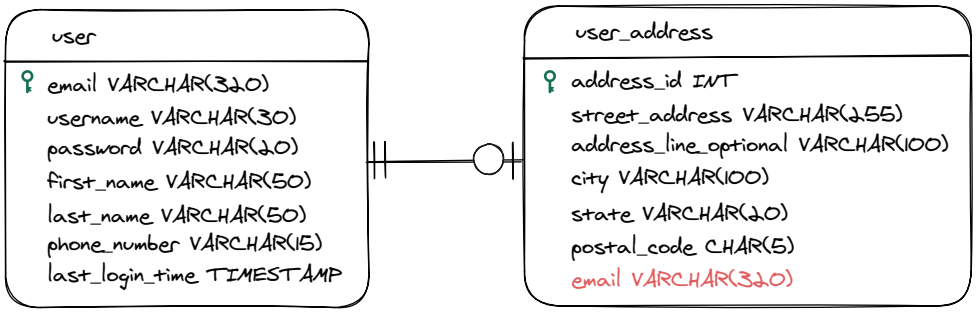
- A user may not have an address when they first register an account, but a user must have one and only one address before making a purchase.
- An address is associated with only a single user.

Based on this information, you can easily determine the min and max cardinalities of the two directional relationships (from `user` to `user_address` and from `user_address` to `user`), and represent the merged relationship accordingly:



Is this relationship one-to-one? Yes. You may notice that the representation is similar to the example featuring departments and managers earlier in this chapter.

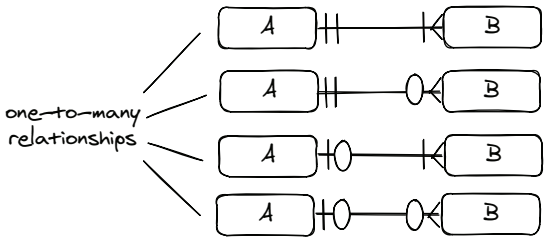
The representation of the relationship between `user` and `user_address` requires the help of a foreign key. As you learned in the preceding section, the min cardinality can indicate where the foreign key should be placed. In the directional relationship from `user` to `user_address`, the min cardinality is zero. Thus, you place the foreign key in the `user_address` entity:



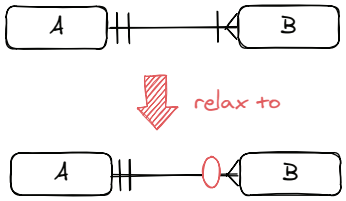
Now you know what a one-to-one relationship is and how to represent it, you should also know that one-to-one relationships are the rarest type because few one-to-one relationships exist in real life.

One-to-many: A love triangle

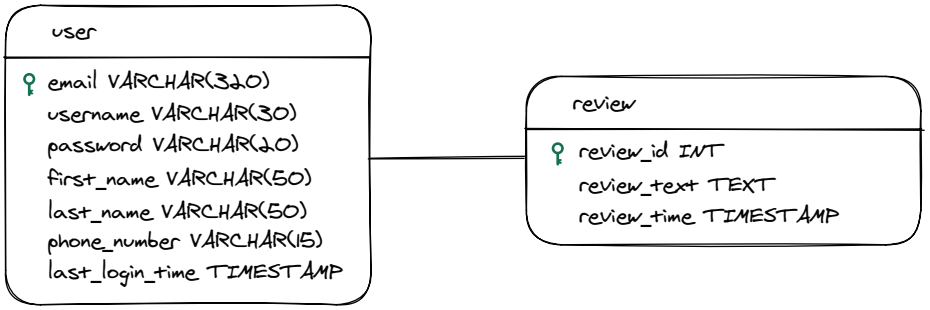
One-to-many relationships are the most common cardinality type. As in one-to-one relationships, *one-to-many* refers primarily to the max cardinalities. If two entities are related, one directional relationship has the max cardinality of one, and the other has the max cardinality of many.



As you saw in the preceding section, if both min cardinalities are ones in one relationship, data entry will be problematic when the representation is implemented. In such a case, you typically relax the min cardinality of the many side from one to zero:



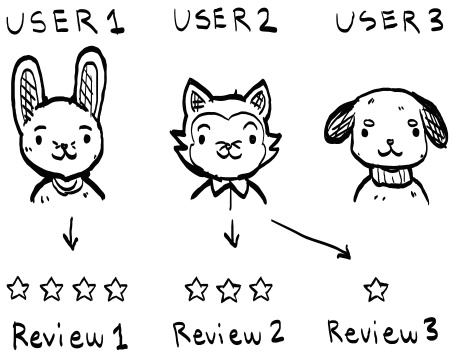
Let's deepen your understanding of a one-to-many relationship by looking at a real example, the relationship between the `user` and `review` entities in the draft diagram of The Sci-Fi Collective:



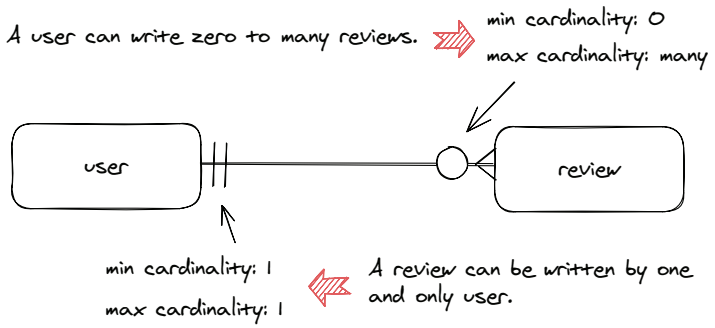
From the requirement-gathering phase and your follow-up inquiry on the relationship between the two entities, you managed to put together the following information:

- A user can write zero to many reviews.
- A review can be written by one and only user.

You can visualize this relationship as follows:



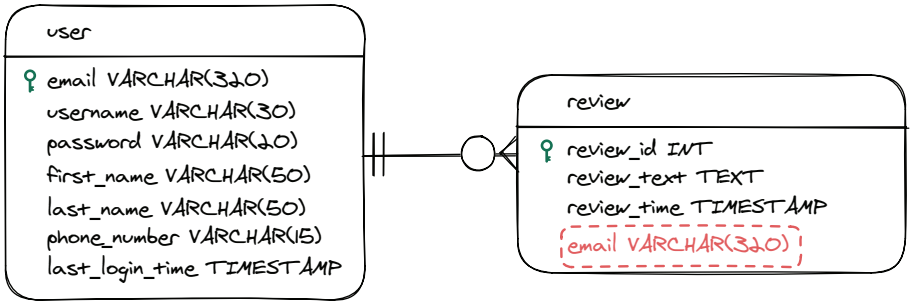
Based on this information, you can determine the min and max cardinalities of the two directional relationships (from `user` to `review` and from `review` to `user`), and represent the merged relationship accordingly:



Categorizing relationships by cardinality has implications for where to place the foreign key, especially in one-to-many relationships. In a one-to-many relationship, the foreign key needs to be placed on the many side—the entity that the crow's foot (→) points to.

Why should the foreign key be placed on the many side of a one-to-many relationship? The many side may have multiple rows that correspond to one row on the one side via the foreign key. If you insist on putting the foreign key on the one side, the one side will need an extra row for each additional record from the corresponding many side, which is not possible.

In our case, the many side is the `review` entity. The foreign key in the `review` entity should be the primary key of the `user` entity. You can update the `review` entity accordingly:



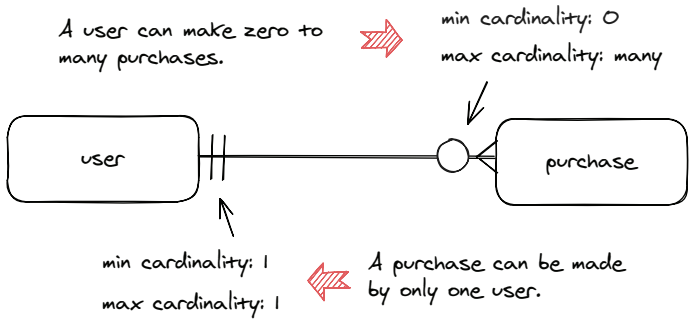
Identify one-to-many relationships in our database

Now you know what one-to-many relationships are, and you have updated the relationship between `user` and `review` entities in the draft diagram. You can apply your new knowledge by identifying other one-to-many relationships and updating them in the draft diagram of The Sci-Fi Collective. Start by reviewing the information you gathered on other relationships:

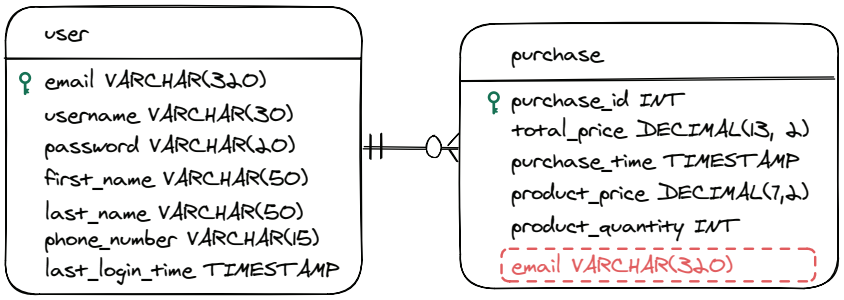
- A user can make multiple purchases. A purchase can be made by only one user.
- A user can maintain multiple payment methods. A payment method can be associated with only one user.
- [new] A payment method can be associated with multiple purchases. A purchase is associated with one payment method.
- A purchase can have more than one product. A product can show up in multiple purchases.

You can easily determine that the `product` and `purchase` relationship shouldn't fall into the one-to-many basket because if you examine its two directional relationships, neither has the max cardinality of one.

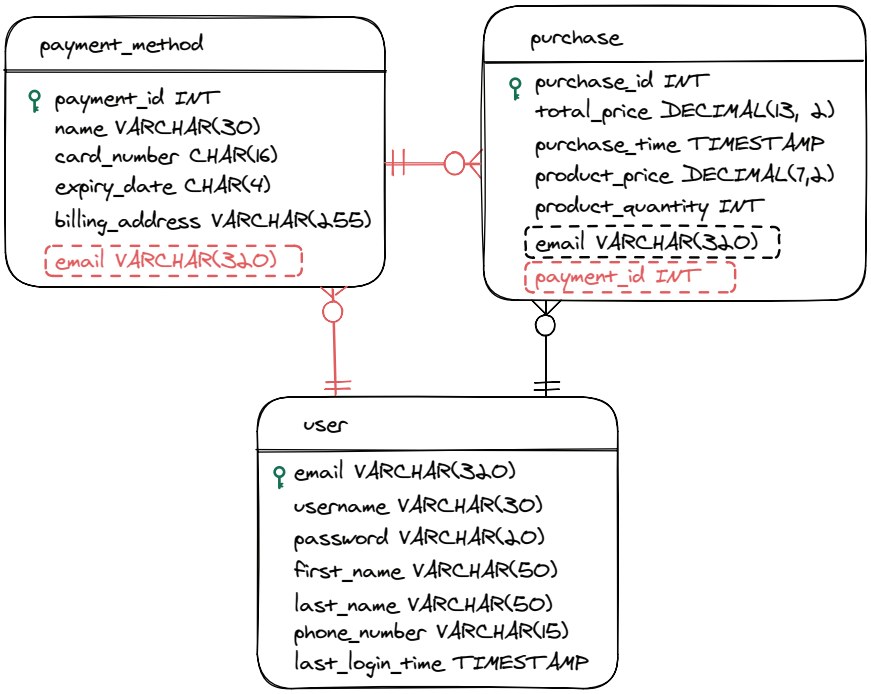
Similarly, it's not difficult to identify the `user` and `purchase` relationship as one-to-many because one of its two directional relationships has the max cardinality of one (from `purchase` to `user`) and the other has the max cardinality of many (from `user` to `purchase`). There is some ambiguity about the min cardinality from `user` to `purchase`, however, because a user can make multiple transactions. But what does *can* mean? After following up with the stakeholders, you learn that the relationship is optional. Think about when you browse an online store for an hour without buying anything. In other words, the min cardinality is zero. Update the `user` and `purchase` relationship as follows:



Next, insert a foreign key on the many side to establish this relationship fully:



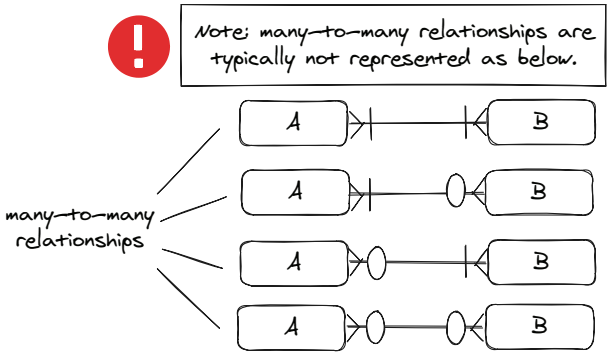
The relationships are one-to-many between `user` and `payment_method` and between `purchase` and `payment_method`. Follow the same steps to update them:



As you can see, you have two sets of relationships going from `payment_method` to `user`; one set goes through the `purchase` entity, and the other is direct. This representation can lead to problems, which we will address in chapter 6.

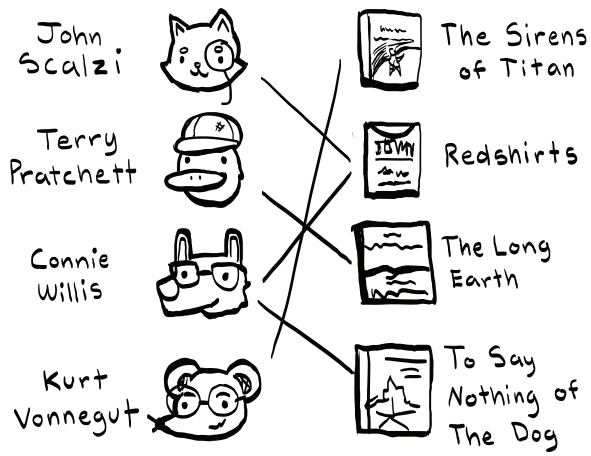
Many-to-many: The more, the merrier

The last cardinality type you will learn is the many-to-many relationship. Like the other two types of relationships, many-to-many refers primarily to max cardinalities. If two entities are related, the max cardinality of both directional relationships is many, as shown in the following figure.

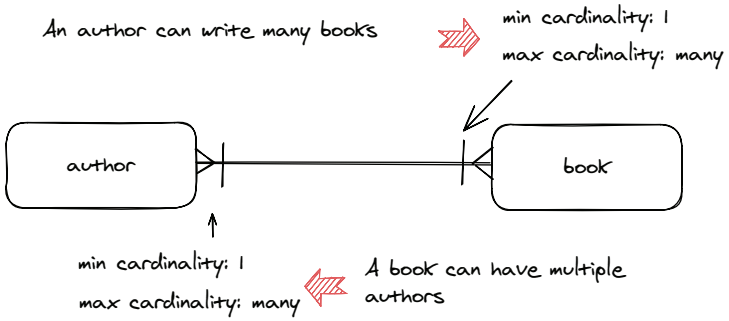


The preceding figure may help you grasp many-to-many relationships, but many-to-many relationships are represented differently. In general, a many-to-many relationship is represented as two one-to-many relationships with the help of a junction table, as we'll demonstrate in an example.

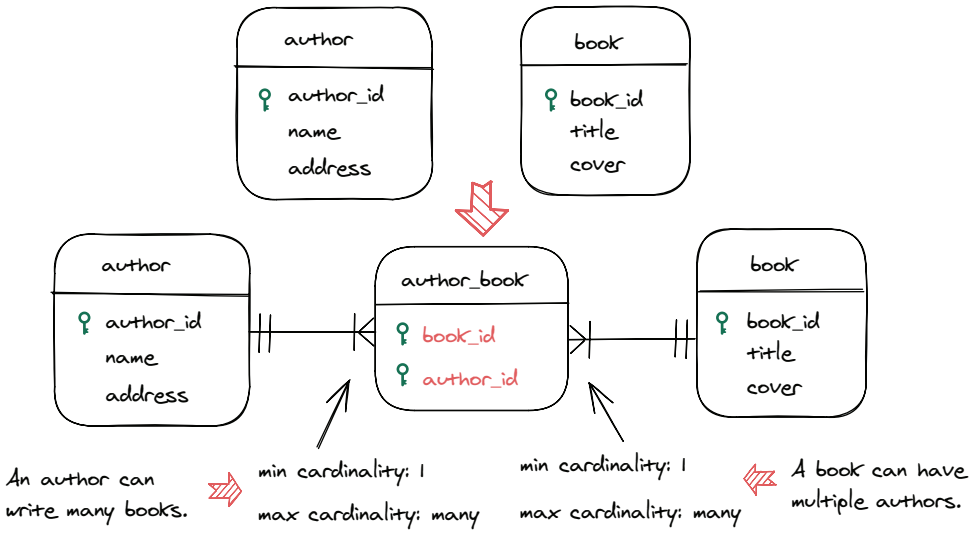
If you design a database for a bookstore application, you need to store all the information about authors and books in two separate tables. You know that an author can write many books and a book can have multiple authors.



You know that this relationship is many-to-many as soon as you identify the max cardinalities of the two directional relationships:



To represent this relationship properly, you need to go one step further by making a junction table that contains only the primary keys of the two involved entities: `author` and `book`. Also, you must convert the many-to-many relationship to two one-to-many relationships between the junction table and both entities. You can represent this many-to-many relationship as follows:



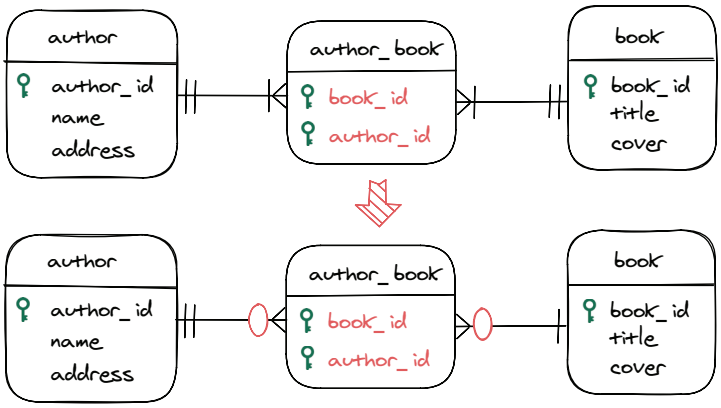
As you see, the junction table, `author_book`, contains only the primary keys from the `author` and `book` entities. In this junction table, the `book_id` and `author_id` attributes together serve as a composite primary key. It's worth noting that the crow's feet always point to the junction table. The `author` and `book` entities become indirectly related via the junction table.

Composite primary keys: The ultimate combination in databases

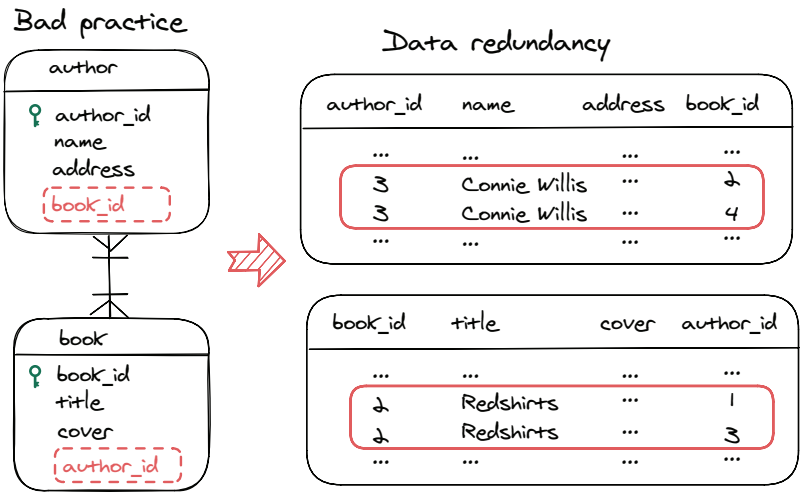
A *composite primary key* is a primary key in a relational database that consists of two or more attributes that together uniquely identify each row in a table. Unlike a primary key based on a single column, a composite primary key requires multiple columns to form a unique identifier for each record.

You must use a composite primary key when a single column cannot guarantee uniqueness for each row but a combination of multiple columns does. This type of key is commonly used when a table has a many-to-many relationship with another table or no simple key can uniquely identify a row in the table.

Now you have two one-to-many relationships whose min cardinalities are all ones. As you saw earlier in this chapter, given such a one-to-many relationship, you need to relax the min cardinality on the many side from one to zero so that you won't face problems in data entry when you implement such a representation.



You may wonder why you can't simply insert a foreign key into both the entities in a many-to-many relationship. Well, representing a many-to-many relationship this way leads to data redundancy and difficulties in querying and modifying data. If you represent the `author` and `book` relationship this way, the data-redundancy problem jumps out immediately:



If an author wrote multiple books, the same author information has to occupy multiple rows in the `author` table even if they are redundant: different books have different `book_id` values, and `book_id` is the foreign key in the `author` table. Similarly, if a book has multiple authors, the information for the same book has to occupy multiple rows in the `book` table: different authors have different `author_id` values, and `author_id` is the foreign key in the `book` table. When the time comes to implement the two tables, having redundant primary key values, such as `author_id` in the `author` table, is impossible. Thus, simply inserting a foreign key into both tables involved in a many-to-many relationship is not viable.

Cardinality yoga: Learning to flex with zeros

The default min cardinality is often set to one. But you need to relax the min cardinalities from one to zero sometimes to enable data entry from time to time:

- In a one-to-one relationship, if one entity is typically created before the other, you might set the minimum cardinality of the second entity to zero to allow the first entity to exist without an immediate counterpart.
- In a one-to-many relationship, you can relax the cardinality that's closer to the many side to allow for cases in which the entity on the one side might exist without corresponding entries on the many side. A customer might exist without any addresses, for example.

(continued)

- In a many-to-many relationship, the minimum cardinalities are often set to zero on both sides if either entity can exist without the other. If there are strict business rules on the order of their existence, you might keep one of the two min cardinalities as one.

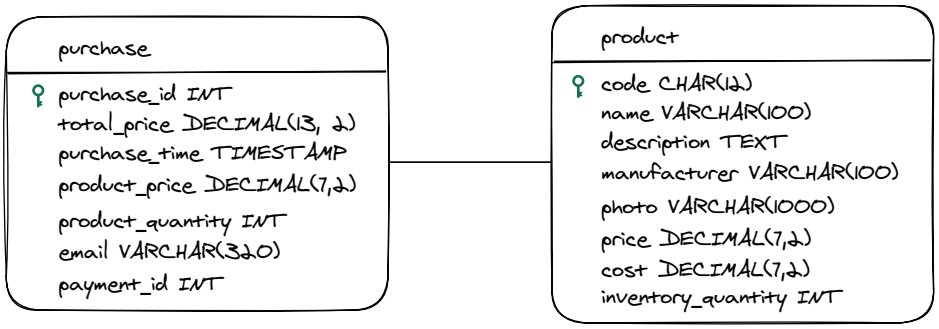
These rules of thumb may not always hold, depending on business rules and requirements, but you can still use them to identify three relationships with min cardinalities that require further examination.

Identify many-to-many relationships in your database

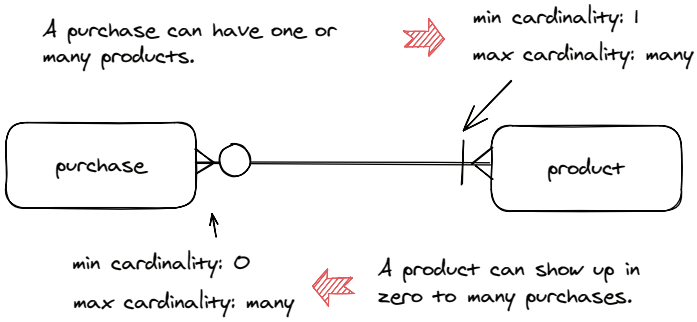
The draft diagram of The Sci-Fi Collective has one more relationship that you haven't worked on yet. This relationship is between the `purchase` and `product` entities:

> A purchase can have more than one product. A product can show up in multiple purchases.

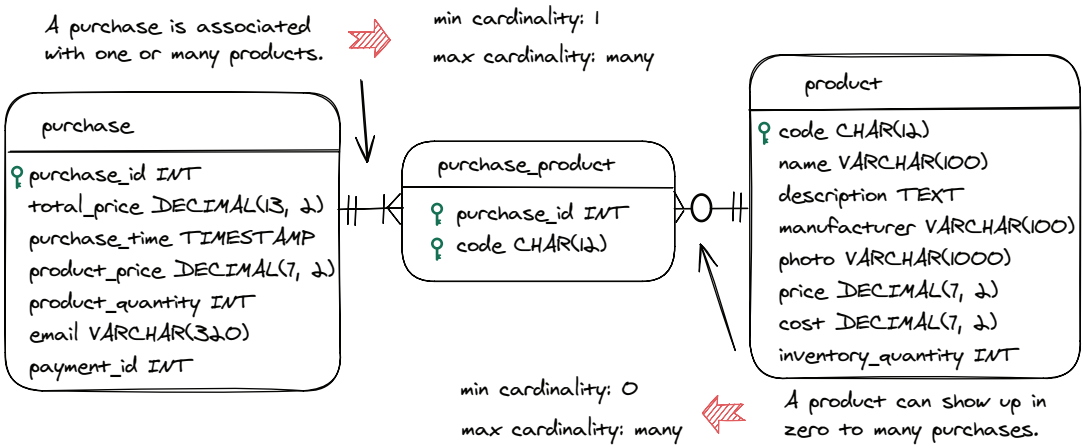
After following up with the stakeholders for clarification, you see that a product doesn't necessarily need to show up in a purchase record, but a purchase record has to be associated with at least one product:



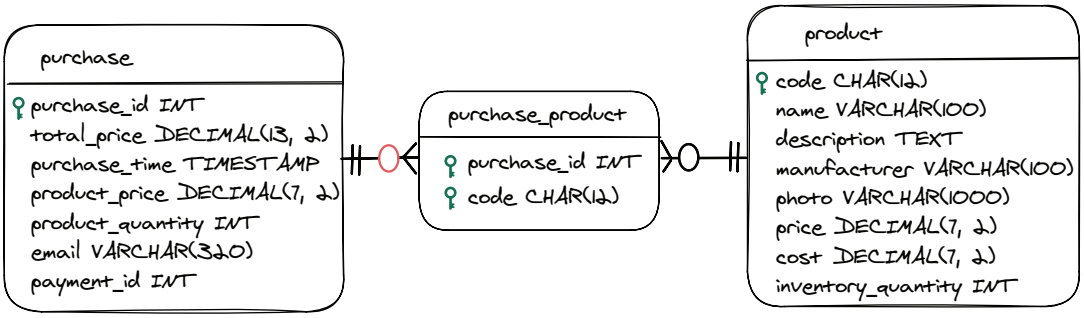
Based on this information, it is easy to identify this relationship as many-to-many and to determine the max and min cardinalities:



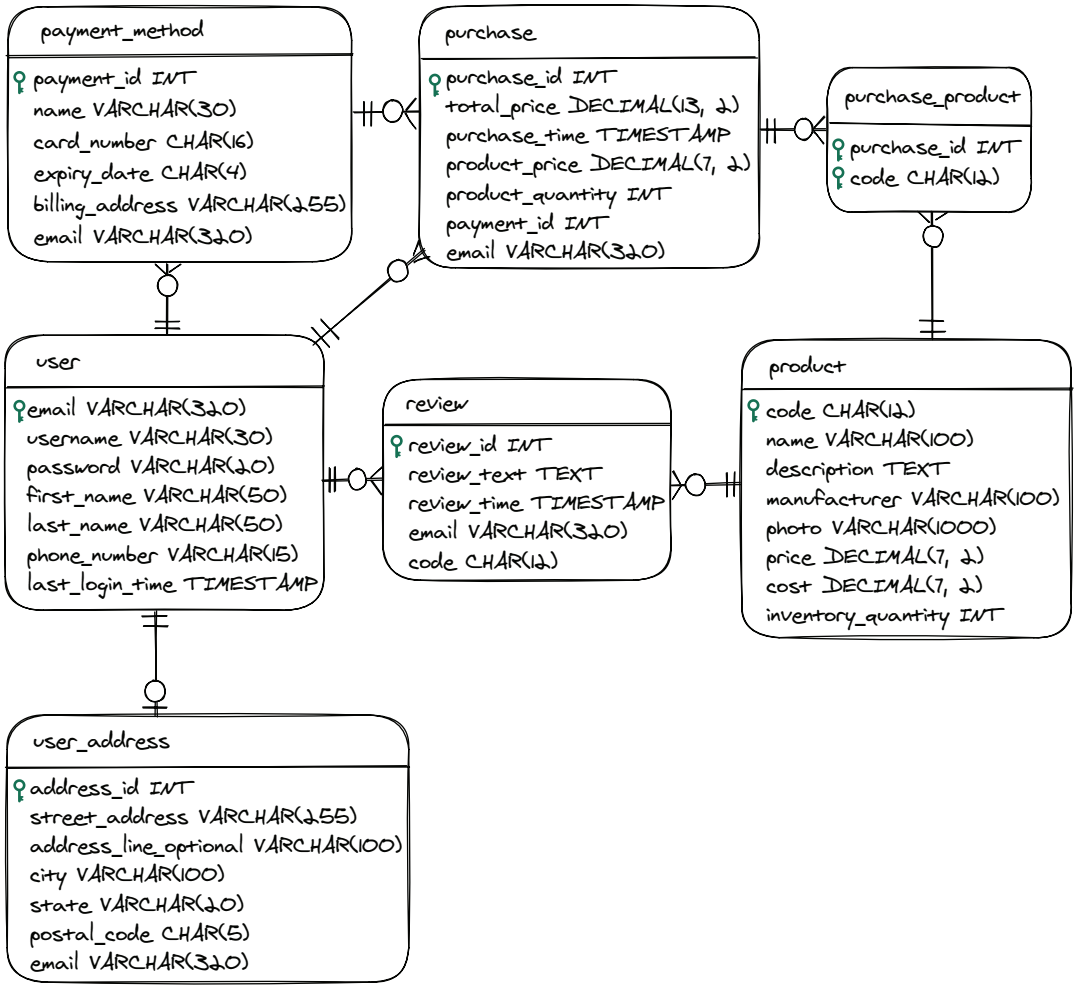
To represent this many-to-many relationship, make a junction table that contains only the primary keys from the `purchase` and `product` entities, and convert the many-to-many relationship to two one-to-many relationships:



You may notice that the min cardinalities of the relationship between `purchase` and `purchase_product` are both one. To prevent problems with data entry when this representation is implemented, you should relax the min cardinality closer to the `purchase_product` side from one to zero, as shown in the following figure.



After all your updates to the draft E-R diagram, it looks like this:

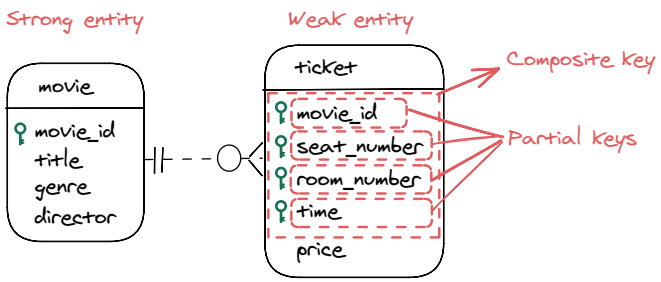


Strong and weak entities

Strong and weak entities are another characteristic of a relationship between two entities. This characteristic may come handy for refining E-R diagrams. In this section, you will learn about strong and weak entities and apply what you've learn to refine the E-R diagram of The Sci-Fi Collective.

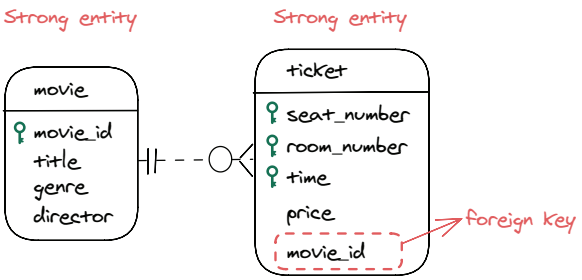
At their core, *strong* and *weak entities* involve a dependency relationship between two entities. Given two related entities, if one is strong and the other is weak, the strong entity can exist on its own, but the weak entity cannot exist without the strong entity. In other words, weak entities cannot be uniquely identified by their attributes alone.

Let's try to understand strong and weak entities better via an example. Suppose that you are designing the database for a movie theater's web application and need to represent the relationship between movies (`movie`) and tickets (`ticket`). Between `movie` and `ticket`, `movie` is the strong entity because `movie` can exist independently in the database. By contrast, `ticket` is the weak entity because it depends on `movie` to identify each of its instances. The following figure depicts this relationship:



Typically, the primary key of the weak entity is a composite key that combines its own attribute(s) and the primary key of the related strong entity. As you see in the preceding figure, `ticket` uses a composite key composed of `movie_id` and other attributes. `movie_id` is the primary key in the `movie` entity, and the other attributes distinguish different tickets to the same movie. Such attributes are also known as *partial keys*.

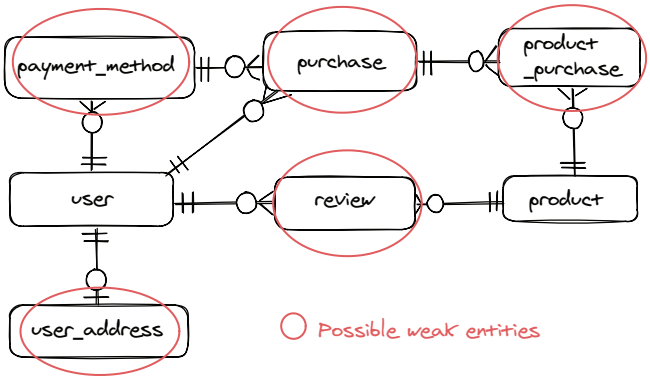
It is worth noting, however, that strong and weak entities can always be converted to two strong entities. If tickets need to be managed independently for reselling and refund purposes, for example, you can convert `ticket` to a strong entity:



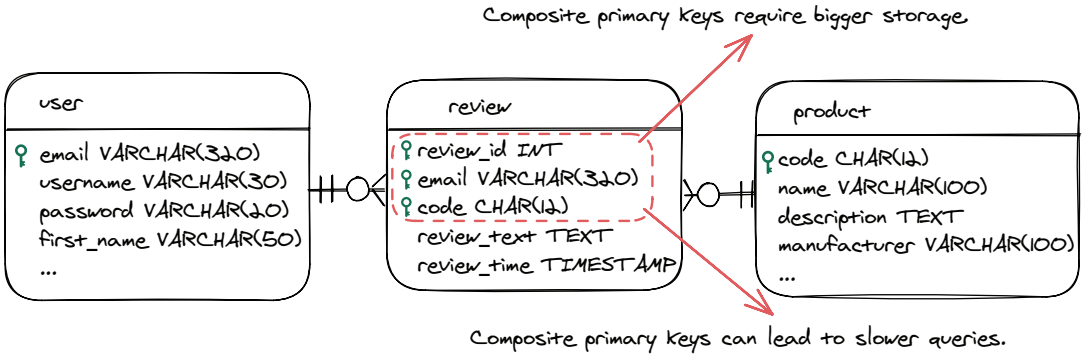
But what you should do if you have two entities that can be identified as both strong and weak? Should you identify them as strong and weak entities or treat them as two strong entities? The answer depends on a variety of factors, such as business demands, data representation and maintenance, and performance. If both options are possible, always choose the one that makes E-R diagrams simpler and less susceptible to bad data. As you apply this new knowledge to refine the E-R diagram of The Sci-Fi Collective further, you will see several examples that involve deciding whether to identify two entities as strong and weak entities.

Identify strong and weak entities in your database

In the E-R diagram of The Sci-Fi Collective, several entities fit the definition of weak entities, such as `review` and `payment_method`. You can see the `review` entity, for example, as a weak entity that depends on the existence of `user` and `product`:



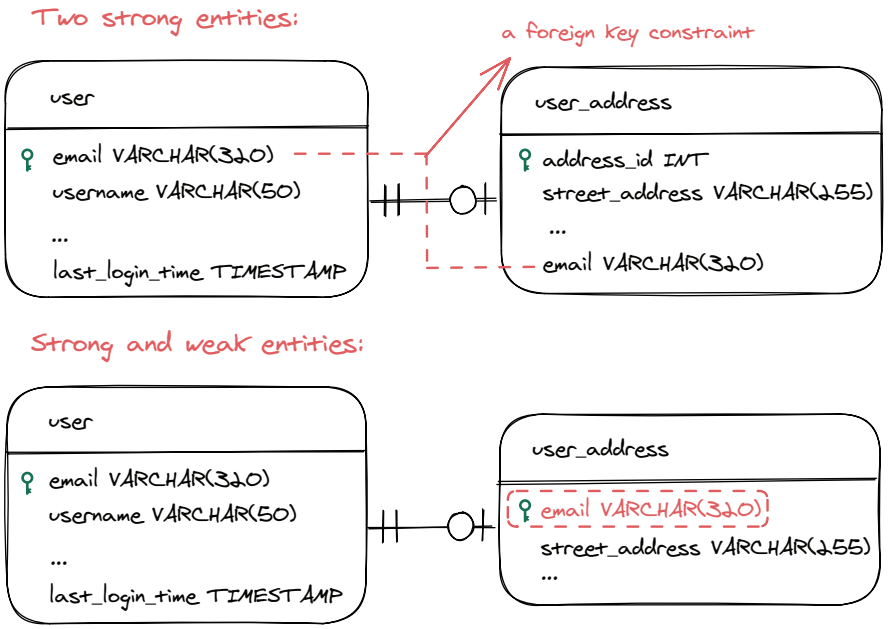
When you consider whether to convert a strong entity to a weak one, you should ask whether the decision will simplify the database design or improve data accuracy. If the answer is no, you should stay put. You could convert `review`, for example, to a weak entity that depends on `user` and `product`. But doing this means you need to start using a composite primary key in the `review` entity, which doesn't introduce any benefits but can lead to increased storage and slower query performance because composite primary keys require wider indexes and slower index lookup. As a result, you shouldn't convert `review` to a weak entity.



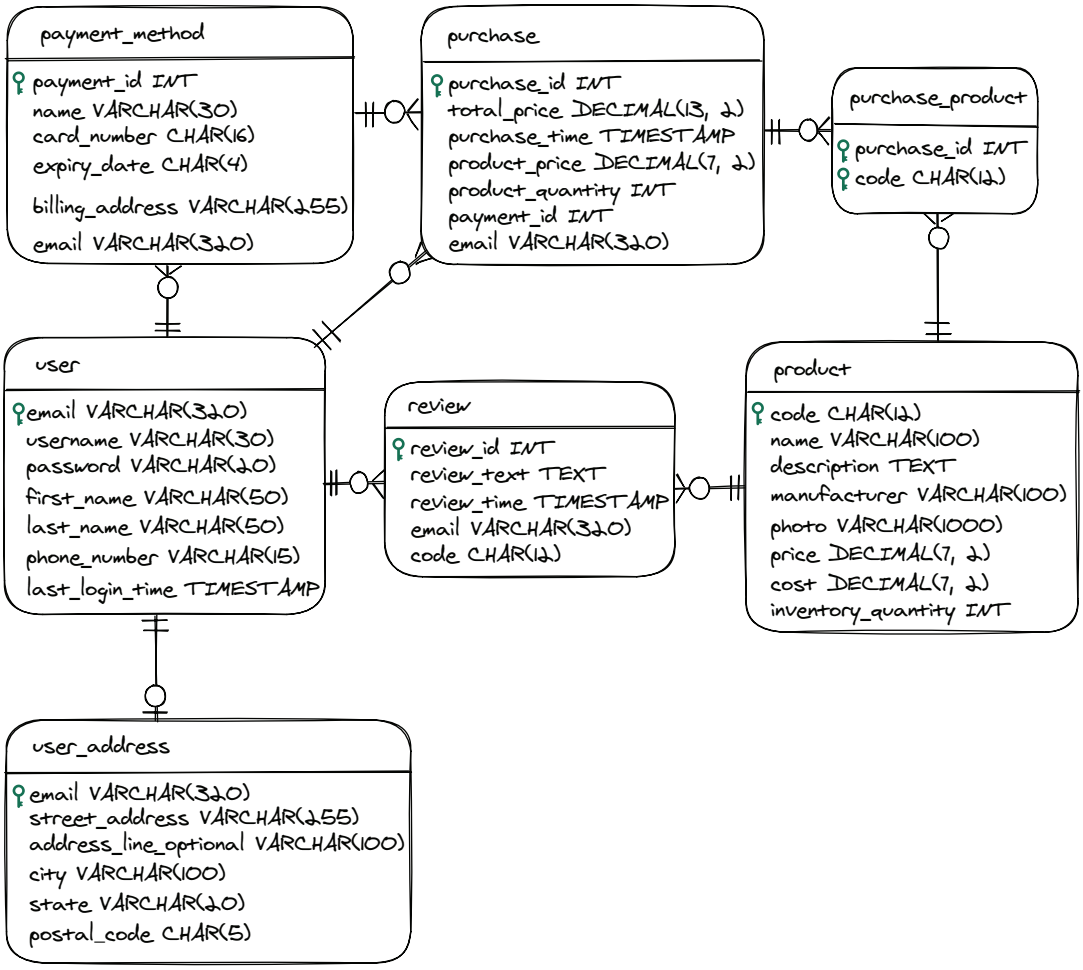
Following the same logic, you can tell that converting `payment_method` or `purchase` to weak entities is a bad idea. What about the `purchase_product` table? You can't change anything about `purchase_product` because it is already a weak entity. The `purchase_product` table is a junction table that you created to link `purchase` and `product` in a many-to-many relationship, and all junction tables are weak entities by nature. In this case, `purchase` and `product` are strong entities. As the weak entity, `purchase_product` uses a composite primary key composed of the primary keys of the two strong entities.

The `user_address` table turns out to be your only opportunity to apply your knowledge of strong and weak entities in refining the draft diagram. Your current design treats `user_address` as a strong entity that uses `address_id` as the primary key to identify each

of its instances and `email` as the foreign key to maintain the link between `user_address` and `user`. If you convert `user_address` to a weak entity, its primary key will become `email`, and the foreign key will be gone. Why? The `user` and `user_address` entities are in a one-to-one relationship, and `email` is good enough to identify each instance of `user_address`. As a result, you no longer need a composite primary key. Converting `user_address` to a weak entity makes sense because it eliminates a foreign key constraint and simplifies your design.

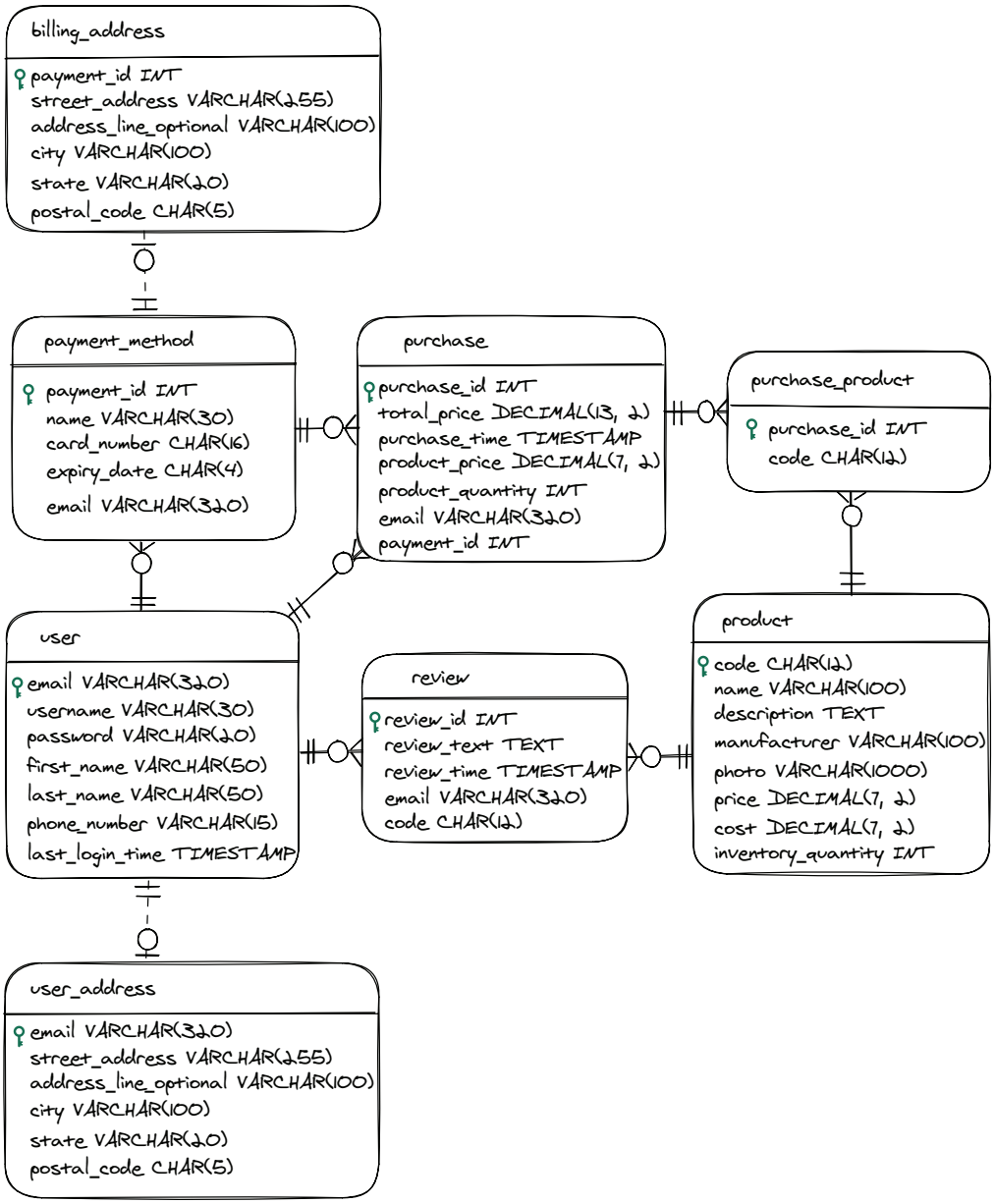


After you convert `user_address` to a weak entity, your E-R diagram looks like this:

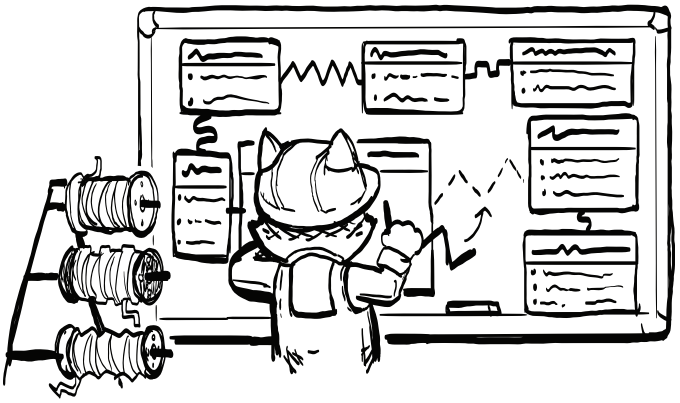


When you take a closer look at other tables and columns, you notice that `payment_method` contains a column, `billing_address`, that represents the billing address of a payment method. The relationship and nuances between a payment method and billing address are the same as between a user and user address. In other words, you need a table to fully represent the billing address the same way that you modeled `user_address`. Following the same logic, you can use a table

called `billing_address` to represent billing addresses. `billing_address` is a weak entity, and its corresponding strong entity is `payment_method`. After you incorporate this change, your E-R diagram looks like this:



Give yourself a pat on the back: you have successfully wrapped up a round of data modeling by developing and refining your E-R diagram. Great job!



Recap

- Relationships are the binding glue between entities. The representations of relationships are informed by information from the requirements-gathering phase and further analysis.
- To represent relationships in an E-R diagram, you typically go through a three-step process: establishing relationships, identifying the cardinality of each relationship, and identifying potential weak entities and determining whether they should be converted to weak entities.
- Cardinality is an important characteristic of a relationship between two entities. Cardinalities come in three flavors: one-to-one, one-to-many, and many-to-many. Different flavors of cardinalities are represented differently, and the flavor has implications for where the foreign key is placed.
- In a one-to-one relationship, both the max cardinalities are one, represented by two bars. The foreign key connecting the two entities can be placed on either side.
- In a one-to-many relationship, the max cardinalities of the two directional relationships are one and many. Many is represented by the crow's foot. The foreign key should be placed on the many side indicated by the crow's foot.
- In a many-to-many relationship, both the max cardinalities of the two directional relationships are many. Representing a many-to-many relationship requires creating a junction table between the two entities and converting the relationship to two one-to-many relationships between each entity and the junction table. The junction table contains the primary keys of both entities.

- Strong and weak entities are another characteristic of a relationship. If two entities can be represented as strong and weak entities, you need to decide whether to do so, depending on whether such a representation makes the E-R diagram simpler and the database more efficient.
- A weak entity uses the primary key of its corresponding strong entity as part of its composite primary key. Other than that, a weak entity is not represented differently in Crow's Foot notation.

Normalization and implementation 6



In this chapter

- You normalize your database design.
- You implement your database design.
- You learn important concepts such as using constraints and cascade.

What you need to know

You can find the database design covered in this chapter implemented in tools commonly used by practitioners, such as dbdiagram.io and MySQL Workbench, in the GitHub repository (https://github.com/Neo-Hao/grokking-relational-database-design). You can navigate to the `chapter_06` folder and follow the instructions in the `README.md` file to load the database design into corresponding tools.

You can also find the SQL scripts corresponding to the almost-finalized database design for different relational database management systems (RDBMSs), including MySQL, MariaDB, PostgreSQL, SQLite, SQL Server, and Oracle.

Overview

In this chapter, you will normalize and implement your database design for The Sci-Fi Collective. By doing so, you will learn about important concepts in database design, such as functional dependency, normalization, and constraints.



Normalization

Before converting your database design to a SQL script that creates the corresponding database and tables, you need to normalize your design. This critical step in database design is known as normalization.

Normalization is the process of organizing a database in a way that minimizes redundancy and dependency while maximizing data integrity and consistency. In other words, we break the database into smaller, more manageable tables, each table representing a single entity or concept. The primary goal of normalization is to strengthen data integrity. Although you have worked toward this goal in the preceding chapters, you are about to kick your work up a notch.